

Titan: Efficient Polynomial Commitments from IOPs over Groups

Abstract

In this paper, we propose Titan, an efficient polynomial commitment scheme (PCS) with transparent setup. It achieves commitment time of $O(n)$, evaluation time of $O(\sqrt{n})$ while the proof size and verification scales as $O(\sqrt{n})$. Titan features an order of magnitude smaller proof sizes than hash based PCS, while featuring a significantly more efficient prover and verifier compared to state of the art group based schemes like Dory and Hyrax. To achieve this balance, Titan borrows two-tiered commitments from Dory, and realizes outer commitment using interactive protocols of proximity (IOPP) over groups, specifically WHIR, instead of expensive bilinear pairings. This allows Titan to be instantiated over general curves with discrete-log hardness such as Pasta Curves, instead of requiring pairing friendly curves.

We compile a variant of Spartan protocol for R1CS with Titan PCS to realize a new SNARK, which we call TitanSnark. Our construction TitanSnark preserves the prover efficiency of the existing Spartan protocol, while improving proof size and verification quadratically from $O(\sqrt{n})$ to $O(\sqrt[4]{n})$. Concretely, for circuits of size $\geq 2^{22}$ this results in around $3\times$ more efficient proof size and verification.

Our blueprint of combining IOPPs over groups with Pedersen style inner commitments is of independent interest, as are several optimizations towards efficiently realizing WHIR IOPP over prime-order groups.

CCS Concepts

• Security and privacy → Cryptography.

Keywords

polynomial commitments, snarks

ACM Reference Format:

. 2018. Titan: Efficient Polynomial Commitments from IOPs over Groups. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 19 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Polynomial commitment schemes (PCS) are pre-eminent primitives in the design of *Succinct Non-Interactive Arguments of Knowledge* (SNARKs). Most recent SNARKs are realized as ideal information theoretic protocol in the form of an *Polynomial Interactive Oracle Proof* (PIOP) which are compiled into an argument of knowledge using a polynomial commitment scheme. Several popular SNARKs

such as Plonk [27], Sonic [39], Marlin [22], Spartan [45], Hyperplonk [21], Brakedown [31], Basefold [49] among others follow the aforementioned modular blueprint. To a great extent, the SNARKs inherit their security and efficiency from the PCS used to instantiate them. For example, whether or not a SNARK requires a trusted or a transparent setup depends on the underlying PCS. Similarly, whether or not a SNARK is plausibly post-quantum safe is also determined by the underlying PCS. Thus, significant body of work exists towards constructing polynomial commitment schemes meeting varied security and efficiency desiderata.

In terms of setup assumptions, arguably, the polynomial commitment schemes with overall practical efficiency require a structured trusted setup. These include the celebrated KZG [35] scheme of Kate, Zaverucha and Goldberg for univariate polynomials and its analogue PST [41] for multilinear polynomials. These schemes are attractive as they feature $O(1)$ and $O(\log n)$ proof sizes and verification respectively. Several variants of KZG scheme such as Gemini [17], Zeromorph [37], Samaritan [30] Mercury [24] have also been subsequently developed for multilinear polynomials. Concrete proof sizes for large polynomials of size 1 million and beyond range from about 0.5 KB-10 KB. A major drawback of the above schemes is their dependence on trusted setup and at times slow cryptography resulting from pairing friendly elliptic curves over which they are instantiated. To mitigate these drawbacks, several approaches have been explored to design PCS from transparent setup, which we discuss below.

A promising line of works including Behemoth [44], Dew [8] and recently DewTwo [20] use groups of unknown order (e.g., class groups) to construct polynomial commitment schemes. While they achieve extremely small proof sizes, currently these schemes are largely of theoretical interest as prover overhead remains practically prohibitive and no experimental evaluation currently exists.

Among the more practical elliptic curve based polynomial commitment schemes are Hyrax [47] and Dory [38]. Both express polynomial evaluation as a vector-matrix-vector product. Using homomorphic Pedersen commitments to rows of the matrix, the evaluation reduces to two inner product relations. Hyrax commits to the polynomial as row commitments of the associated matrix, which gives it a large $O(\sqrt{n})$ commitment size and verification complexity. Dory improves upon the approach in Hyrax by using bilinear pairings to construct a two-tiered commitment as a commitment over row commitments. It then uses a novel inner product argument with logarithmic verification complexity to realize PCS with $O(\log n)$ verifier, while featuring $O(n)$ commitment cost and $O(\sqrt{n})$ evaluation. Concretely, Dory achieves proof sizes in the range of 20KB-30KB for large polynomials. While featuring essentially optimal asymptotics, Dory employs bilinear pairings and target group operations which result in somewhat slower prover and verifier. It also fails to leverage non pairing-friendly elliptic curves with faster cryptographic operations.

Another popular approach to designing polynomial commitments involves compiling information theoretic protocols into an argument using hash functions. At a high level, these schemes

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

encode a polynomial as a codeword in a linear code and prove evaluation by showing proximity of an associated vector to a linear code via the *interactive oracle proofs of proximity* (IOPP). The proximity claim is recursively reduced in several rounds, till the claim involves vectors small enough to be verified directly. The protocols use Merkle trees based on collision resistant hash functions (CRHF) to realize polynomial oracles. These constructions feature entirely symmetric key cryptography which makes them prover and verifier efficient in addition to being plausibly post-quantum safe. The prominent schemes using this hash based paradigm include FRI [11], DEEP-FRI [15], Brakedown [31], Basefold [49], STIR [4] and recent works WHIR [5] and DeepFold [32]. On the downside, the proofs are often quite large ranging from few hundred kilobytes to few megabytes. The proof sizes are even larger when they are based on more conservative (provable) bounds, instead of aggressive (conjectured) bounds which have recently been disproved in [23].

1.1 Our Contributions

Against this impressive body of work, we contribute a new transparent polynomial commitment scheme Titan, and a variant of Spartan [45] compiled with Titan PCS, which we call TitanSnark. The key objective of Titan is to achieve order of magnitude smaller proof sizes than hash based schemes, while achieving significantly more efficient prover and verifier than group based schemes. Titan achieves this balance by avoiding expensive elliptic curve operations such as bilinear pairings and target group operations, while incorporating symmetric key based techniques from hash based constructions. These benefits also translate to TitanSnark, which improves the transparent variant of Spartan both asymptotically and concretely. We summarise our contributions below:

- We construct transparent polynomial commitment scheme Titan with constant commitment size, $O(n)$ commitment time, $O(\sqrt{n})$ evaluation time, $O(\sqrt[3]{n})$ proof size and $O(\sqrt[3]{n})$ verification time. The scheme can be instantiated over both pairing friendly and non-pairing friendly curves with smooth subgroups of size $O(\sqrt{n})$. Our scheme can be used both with univariate and multilinear polynomials.
- When instantiated over Pasta Curves (which have efficient group operations), Titan achieves 2.5× faster commitment time, 10× faster evaluation time and 4× faster verification than Dory, while featuring competitive proof size. Titan also compares favourably against Hyrax in most metrics. At the cost of giving up post-quantum security, Titan achieves an order of magnitude smaller proofs than hash based schemes. Titan also features extremely fast evaluation proofs, being $O(\sqrt{n})$ asymptotically without expensive pairings. Concretely, for polynomials of size 2^{20} , Titan achieves proof sizes of 25 KB, which can be verified in under 50 millisecond with a single threaded implementation. Additional evaluation appears in Section 7.
- We use Titan PCS to design a new variant of Spartan, called TitanSnark which retains the prover efficiency of the original scheme in [45], while reducing proof size and verification complexity from $O(\sqrt{n})$ to $O(\sqrt[3]{n})$. Concretely, this leads to about 3× smaller proof size for polynomials of size $\approx 2^{23}$. Additional benchmarks are provided in Table 4.

Towards the goal of designing Titan with above efficiency profile, we make several technical contributions of independent interest.

- We adapt the two-tiered commitments by swapping expensive bilinear pairings for the outer commitment with IOPP over group polynomials. We then adapt techniques from multilinear IOPPs such as Basefold [49], WHIR [5] to work over prime-order groups instead of fields. The broad framework for sum-check based PIOPs over groups has been explored in prior work [18], while more recent applications to optimize verifier in inner product arguments (IPA) appear in [25, 50]. Nevertheless, our application towards realizing two-tiered commitments is novel to the best of our knowledge and contributes additional techniques.
- We propose several optimizations to efficiently enable multilinear IOPPs over groups. A key optimisation we introduce is adapting sumcheck over group polynomials arising in Basefold/WHIR IOPPs to exclusively use multi-scalar multiplications (MSMs) instead of the expensive group-exponentiations. This crucially makes our evaluation proofs extremely fast.
- We implement our scheme in Rust and provide a thorough experimental evaluation and comparison against relevant state of the art schemes.

2 Technical Overview

We present an overview of our techniques here. First, in Sec.2.1, we describe Titan, our multilinear PCS. In particular, we explain why we need to tailor and generalize multilinear IOPPs to groups. Multilinear IOPPs such as Basefold [49], WHIR [5] rely on sumcheck protocol and, therefore, key to “group-based IOPPs” is making sumcheck protocol work *efficiently* over groups – this we discuss in Section 6.2. Detailed constructions with security proofs appear in Sections 4, 5 and Section B in Appendix. Finally, we describe how to construct a SNARK for R1CS by combining Titan with MicroSpartan [51] PIOP (a recent variant of Spartan [45]) for R1CS. The description of our SNARK can be found in Section D in Appendix.

2.1 Titan PCS

We describe Titan PCS in the setting of IOPs. In this setting, we view the function oracle $\llbracket f \rrbracket$ as *commitment* to function over a domain. In particular, the prover commits to a polynomial via the function defined by its evaluation over the domain. In the *evaluation* phase, verifier receives the function oracle $\llbracket f \rrbracket$ as purported commitment to a polynomial, the evaluation point z , and the claimed evaluation σ . The evaluation IOP accepts if and only if σ is the correct evaluation of polynomial defined by f at z . Applying BCS [14] transformation we obtain PCS in the *Random Oracle Model* (ROM), where function oracles are realized using vector commitments with succinct opening proofs, and verifier challenges are obtained using random oracle queries. We defer precise definitions to Section 4.

2.1.1 $O(\sqrt{n})$ construction. We first describe a simplified version of Titan PCS that only achieves $O(\sqrt{n})$ proof-size and verifier time, but conveys the core ideas. At the end of the section we outline how the simplified scheme can be tweaked to reduce proof-size and verifier time to $O(\sqrt[3]{n})$, by reorganizing the oracle access. In the following discussion, $\mathbb{F}$ refers to the scalar field of prime order p with a smooth domain $L \subseteq \mathbb{F}$, and $\mathbb{G}$ refers to a group of order p in which the discrete-log problem is computationally hard.

The polynomial commitment scheme for multilinear polynomials in m -variables over $\mathbb{F}$ consists of:

- a *setup* algorithm, which samples a vector $\mathbf{g} = (g_1, \dots, g_q)$ with $q = 2^{m/2} = \sqrt{n}$ and each g_i uniformly sampled from $\mathbb{G}$.
- a *commit* algorithm, which is used by a prover $\mathcal{P}$ to commit to a polynomial $\tilde{f}(\mathbf{Z}) \in \mathbb{F}^{\leq 1}[Z_1, \dots, Z_m]$ in at most m variables as an oracle $\llbracket f \rrbracket$; and
- an interactive *evaluation* protocol, which is used by $\mathcal{P}$ to convince a verifier $\mathcal{V}$ with oracle access $\llbracket f \rrbracket$ that $\tilde{f}$ committed by it evaluates to σ at $\mathbf{z}$.

At a high level, $\mathcal{P}$ generates a commits to $\tilde{f}$ as follows:

- (1) Decompose $\tilde{f}$ into q multilinear polynomials $\tilde{f}_1(\mathbf{Y}), \dots, \tilde{f}_q(\mathbf{Y})$, each in $m/2$ variables $\mathbf{Y} := (Y_1, \dots, Y_{m/2})$, where $\tilde{f}_i(\mathbf{Y}) := \tilde{f}(\langle i \rangle, \mathbf{Y})$ is $\tilde{f}$ partially evaluated at $\langle i \rangle$, the bit decomposition of i .
- (2) Use commitment key $\mathbf{g}$ to generate Pedersen commitment $G_i \in \mathbb{G}$ to each row polynomial $\tilde{f}_i(\mathbf{Y})$, i.e. $G_i = \sum_{j=1}^q \tilde{f}_i(\langle j \rangle) \cdot g_j$.
- (3) Let $\tilde{G}(\mathbf{X}) \in \mathbb{G}[X_1, \dots, X_{m/2}]$ be the $m/2$ -variate group polynomial obtained by interpolating the group elements $G_1, \dots, G_q$ over $\{0, 1\}^{m/2}$. Output oracle $\llbracket G \rrbracket$ to commit to the polynomial $\tilde{G}(\mathbf{X})$. We treat $\llbracket G \rrbracket$ as a commitment to the polynomial $\tilde{f}$.
- (4) Additionally, let $\tilde{g}(X_1, \dots, X_{m/2})$ be the polynomial interpolating the generators. We assume that the verifier has oracle access $\llbracket g \rrbracket$ which commits to the polynomial $\tilde{g}^1$.

Next, to prove that $\tilde{f}(\mathbf{z}) = \sigma$, where $\mathbf{z} \in \mathbb{F}^m$ is parsed as $\mathbf{x}, \mathbf{y} \in \mathbb{F}^{m/2}$, $\mathcal{P}$ and $\mathcal{V}$ proceed as follows, with $\mathcal{V}$ accepting in the end only if all the checks pass:

- (1) $\mathcal{P}$ computes the partially-evaluated polynomial $\tilde{a}(\mathbf{Y}) := \tilde{f}(\mathbf{x}, \mathbf{Y})$, computes its (Pedersen) commitment C and sends it to $\mathcal{V}$.
- (2) $\mathcal{P}$ and $\mathcal{V}$ execute the evaluation protocol of group-based multilinear IOPP to ensure that $\tilde{G}(\mathbf{x}) = C$, where $\tilde{G}$ is purported polynomial committed by oracle $\llbracket G \rrbracket$.
- (3) $\mathcal{P}$ uses a proof of knowledge, concretely Compressed Sigma Protocol (CSP) [9], to convince $\mathcal{V}$ that it *knows* the partially-evaluated polynomial $\tilde{a}(\mathbf{Y})$ for the commitment C and that $\tilde{a}(\mathbf{y}) = \sigma$.

On a high level, binding follows as long as $\llbracket G \rrbracket$ binds $\mathcal{P}$ to unique $\tilde{G}$ (later we use IOPs of Proximity to ensure this), while $\tilde{G}$ in turn binds $\mathcal{P}$ to unique $\tilde{f}$ due to binding property of the Pedersen commitment. Let us now elaborate a bit more on how the simplified scheme works.

Committing, in two stages. Recall that $q = 2^{m/2} = \sqrt{n}$. For $i \in [q]$, let $\tilde{e}q_i(\mathbf{X}) := \tilde{e}q(\mathbf{X}, \langle i \rangle)$ denote i th Lagrange basis polynomial for $\mathbb{F}^{\leq 1}[X_1, \dots, X_{m/2}]$. As in prior works such as Hyrax [47] and Dory [38], we view $\tilde{f}(\mathbf{Z})$ as a $q \times q$ matrix (over $\mathbb{F}$) where the i th row consists of the q (canonical) evaluations of the $m/2$ -variate multilinear polynomial $\tilde{f}_i(\mathbf{Y}) = \tilde{f}(\langle i \rangle, \mathbf{Y})$. More formally, this decomposition can be written as:

$$\tilde{f}(\mathbf{X}, \mathbf{Y}) = \sum_{i=1}^q \tilde{e}q(\mathbf{X}, \langle i \rangle) \cdot \tilde{f}_i(\mathbf{Y}). \quad (1)$$

¹Since the generators are public, we assume that the verifier computes the correct encoding of $\tilde{g}$ one-time. Since the encoding is of size $O(\sqrt{n})$, we assume it is locally stored by the verifier

At this point, instead of using a “two-stage” commitment that relies on computationally-expensive bilinear maps as in Dory, we design and use a more efficient two-stage commitment based on Pedersen commitment and IOPP over groups. It is this switch that allows us to instantiate $\mathbb{G}$ even with plain groups, which *need not* support pairings, leading to efficiency gains. Next we describe our two-tiered commitment.

Recall that $\mathbf{g} = (g_1, g_2, \dots, g_q) \in \mathbb{G}^q$ is the vector of uniformly-sampled generators, serving as the key to Pedersen commitment. $\mathcal{P}$ uses Pedersen commitment to commit to $\tilde{f}_i$ s, the partial evaluations of $\tilde{f}$ over the hypercube $\{0, 1\}^{m/2}$. That is, the commitment to the polynomial $\tilde{f}_i \in \mathbb{F}^{\leq 1}[X_1, \dots, X_{m/2}]$ is:

$$G_i = \text{Com}(\tilde{f}_i; \mathbf{g}) = \sum_{j=1}^q \tilde{f}_i(\langle j \rangle) \cdot g_j. \quad (2)$$

At this point there are q group elements $G_1, \dots, G_q$, which $\mathcal{P}$ interpolates into a multilinear polynomial $\tilde{G}(\mathbf{X})$ with coefficients over $\mathbb{G}$, i.e., $\tilde{G}(\mathbf{X})$ is the unique polynomial such that $\tilde{G}(\langle i \rangle) = G_i$ for all $i \in [q]$. The oracle $\llbracket G \rrbracket$ for $\tilde{G}$ serves as the eventual commitment to $\tilde{f}$. To this end, we now review multilinear IOPs and explain their generalization to multilinear polynomials over groups.

Overview of Multilinear IOPs of Proximity. We briefly recall the general blueprint for realizing a (univariate) PCS from an IOPP for Reed-Solomon Code $\text{RS}[\mathbb{F}, L, m]$ of degree $2^m - 1$ and evaluation domain $L \subseteq \mathbb{F}$:

- To commit to a polynomial $\hat{f}$ of degree $d := \rho|L|$, where ρ is the rate of the code, $\mathcal{P}$ commits to its evaluations $\mathbf{f} \in \mathbb{F}^{|L|}$ over L .
- To verify a claim $\hat{f}(v) = \sigma$, $\mathcal{P}$ and $\mathcal{V}$ run an IOPP to check that the quotient function $q(x) = (\hat{f}(x) - \sigma)/(x - v)$ is close to a polynomial of degree at most $(d - 1)$.

In Basefold [49] and subsequently in WHIR [5], the above approach was extended to multilinear polynomials. Our presentation here is more aligned with that in [5] which specifically deals with Reed-Solomon encodings. WHIR IOPP exploits a natural correspondence between m -variate multilinear polynomials and degree- 2^m univariate polynomials: the univariate polynomial $\hat{f}(X)$ corresponding to $\tilde{f}(\mathbf{X})$ is defined as

$$\hat{f}(X) := \tilde{f}(X, X^2, \dots, X^{2^{m-1}}). \quad (3)$$

Given this correspondence, WHIR enables commitments to multilinear polynomials by reducing their evaluation-claims to checking proximity to *constrained* Reed-Solomon codes $\text{CRS}[\mathbb{F}, L, m, \tilde{w}, \sigma]$ which is the set of (evaluations of) univariate polynomials $\hat{f} \in \mathbb{F}^{\leq 2^m}[X]$ whose corresponding multilinear polynomial $\tilde{f}$ satisfies

$$\sum_{\mathbf{b} \in \{0,1\}^m} \tilde{w}(\tilde{f}(\mathbf{b}), \mathbf{b}) = \sigma,$$

where $\tilde{w}$ is an $(m + 1)$ -variate polynomial².

²In this paper, we only consider multilinear $\tilde{w}$

Group-based Multilinear IOPPs. We now describe generalization of multilinear IOPPs to groups. To this end, first we define Reed Solomon encoding over groups $\text{RS}[\mathbb{G}, L, m]$ to be the set of functions $g : L \rightarrow \mathbb{G}$ such that there exists a polynomial $\widehat{G} \in \mathbb{G}[X]$ of degree $< 2^m$ such that $g(x) = \widehat{G}(x)$ for all $x \in L$. Then we show that both $\text{RS}[\mathbb{F}, L, m]$ and $\text{RS}[\mathbb{G}, L, m]$ have similar distance properties, which is necessary for folding to work. Next, to enable evaluation, we need to make sumcheck and folding work for $\text{RS}[\mathbb{G}, L, m]$. While the folding ports naturally to groups, making sumcheck protocol work efficiently over groups requires care. In [18], Bootle et al. generalized the classical sumcheck protocol to polynomials defined over modules. In our setting we view $\mathbb{G}$ as a vector space over the field $\mathbb{F}$. Under this interpretation, the evaluation of a polynomial over groups is well-defined and we can define multiplication with field polynomials by taking scalar multiplication of the coefficients. Consequently, the algebraic structure required for sumcheck naturally extends to polynomials over $\mathbb{G}$. While this naive version of group sumcheck works, it incurs expensive group exponentiations. In Section 6.2, we describe a more efficient sumcheck over groups that uses *multi-scalar-multiplications* (MSMs) instead of $O(n)$ group exponentiations³. With all these group machinery in place, Basefold/WHIR extend to group-based polynomials, and thus $\mathcal{P}$ (in Titan) does the following in Step 3 of the commit phase (Step 4 is similar):

- (1) Encode the group polynomial $\widehat{G}$ as a Reed Solomon codeword using the correspondence from Eq.3, i.e., as evaluations of the associated degree- $\sqrt{n}$ univariate group polynomial $\widehat{G}(X) = \widehat{G}(X, X^2, \dots, X^{\sqrt{n}-1})$ over L . Since $\widehat{G}$ is a $\sqrt{n}$ -sized polynomial, the size of the (codeword) oracle is also $O(\sqrt{n})$ (group elements).
- (2) When compiling to a non-interactive argument in the Random Oracle Model, use a Merkle tree based on a collision resistant hash function $\mathcal{H} : \mathbb{G}^* \rightarrow \{0, 1\}^{2\lambda}$ to succinctly commit to encoding of $\widehat{G}$ thereby realizing the oracle $\llbracket G \rrbracket$.

We refer the reader to Sec.4.2 for further details.

How to evaluate. Finally, to evaluate a claim $\tilde{f}(\mathbf{x}, \mathbf{y}) = \sigma$, we proceed in two stages. In the first stage, the prover proves $C \in \mathbb{G}$ to be the evaluation $\widehat{G}(\mathbf{x})$ of the polynomial $\widehat{G}$ encoded by the oracle $\llbracket G \rrbracket$. Note that in the honest case, C is also the Pedersen commitment of the partially evaluated polynomial $\tilde{a}(\mathbf{Y}) = \tilde{f}(\mathbf{x}, \mathbf{Y})$. Next, the prover shows knowledge of the field polynomial $\tilde{a}(\mathbf{Y})$ which opens the commitment C and satisfies $\tilde{a}(\mathbf{y}) = \sigma$. This latter proof of knowledge uses the *Compressed Sigma Protocol* (CSP) framework [9]. Thus, the evaluation protocol works as follows:

- (1) $\mathcal{P}$ computes the partially evaluated polynomial $\tilde{a}(\mathbf{Y}) = \tilde{f}(\mathbf{x}, \mathbf{Y})$, and a Pedersen commitment $C = \text{Com}(\tilde{a}; \mathbf{g})$ to it using the commitment key $\mathbf{g}$. $\mathcal{P}$ sends C to $\mathcal{V}$.
- (2) $\mathcal{V}$ checks, using multilinear IOPP over groups that $\widehat{G}(\mathbf{x}) = C$ for $\widehat{G}$ encoded by $\llbracket G \rrbracket$.

- (3) $\mathcal{P}$ shows knowledge of field polynomial with coefficients $\mathbf{a} \in \mathbb{F}^q$ via a *proof of knowledge* for the relation:

$$\left\{ (C, \sigma, \mathbf{g}; \mathbf{a}) : \sum_{i=1}^{\sqrt{n}} a_i \cdot g_i = C \wedge \sum_{i=1}^{\sqrt{n}} a_i \cdot \tilde{eq}(\mathbf{y}, \langle i \rangle) = \sigma \right\}$$

Note that the CSP incurs a communication and verification complexity $O(\sqrt{n})$. Also note that the above IOPP only ensures that $\llbracket G \rrbracket$ encodes a word which is close to a Reed Solomon codeword – for a PCS we need to bind G to a unique polynomial. This can, however, be resolved using standard approaches of ensuring proximity of the function to *unique* polynomial or using *out-of-domain sampling* to leverage larger proximity parameters. We refer the reader to Sec.5 for the formal details.

Obtaining $O(\sqrt[4]{n})$ proof-size and verification time. The simplified Titan PCS we just described yields a realization of a transparent multilinear PCS to a polynomial $\tilde{f}$ of size n with $O(\sqrt{n})$ sized oracle. To bring down the proof-size and verifier time to $O(\sqrt[4]{n})$, we rely on a couple of common optimization tricks:

- (1) First, we exploit the fact that IOPPs like Basefold/WHIR query the oracle over sets structured as cosets of a subgroup. Thus we commit to evaluated polynomials *coset-wise*. In more detail, instead of Merkle committing to a polynomial $\tilde{f}$ by setting the leaves to be *individual* evaluations, we first partition the evaluation domain L into appropriate-sized cosets and then Merkle commit with leaves corresponding to evaluations *over these cosets*. For best performance, we set the coset size to be roughly $\sqrt[4]{n}$, which results in a Merkle tree of depth $\log(\sqrt[4]{n})$ with $O(\sqrt[4]{n})$ leaves.
- (2) Naively, a CSP verifier over instance of size $\sqrt{n}$ incurs $O(\sqrt{n})$ costs. Prior works such as [19, 28, 29] have obtained more efficient CSP verifiers in *trusted setup*, by having structured commitment keys. More recently, in [25, 50], sublinear verifier effort for *inner product arguments* (IPA) in transparent setting has been achieved based on the following observation: generators in each round of an IPA can be viewed as a polynomial in evaluation basis, obtained by folding the polynomial corresponding to original generators by IPA verifier's challenges. This allows delegating the folding to the prover, and have it send the folded generators once the instance size decreases to $o(n)$. The verifier can then check the correctness of reduced instance, by querying the original polynomial. In Titan also, we encode Pedersen generators as multilinear oracle and terminate the CSP when the folded instance size becomes $\sqrt[4]{n}$, giving us a protocol with $O(\sqrt[4]{n})$ communication and verification complexity.

Optimizing IOPPs over Groups. We discuss several optimizations to efficiently realize IOPPs over groups. In Section 6, we describe optimizations targeting efficiency of encoding group oracles, executing sumcheck protocol over groups arising in multilinear IOPPs and efficiently computing round messages in a CSP.

TitanSnark In Section D, we combine Titan PCS with Spartan [45] PIOP for R1CS satisfiability to obtain a SNARK for R1CS. We improve upon the transparent version of Spartan (Spartan_{DL} in [45]), reducing proof sizes and verification time (both $\sqrt[4]{n}$ instead of $\sqrt{n}$)

³Specifically, it uses $O(\sqrt{n})$ MSMs each of size $O(\sqrt{n})$

while featuring comparably performant prover⁴. The concrete comparison is presented in Section 7, and further details deferred to Appendices D and E.

Comparison with Recent Group Based IOPPs. We end the overview with a comparison of our techniques with very recent related works [25, 50]. These works also use group based IOPPs to obtain sublinear verification for IPAs. However, in both the aforementioned works, the group-based IOPP works over public polynomials of size $O(n)$ – with significant cost to the prover. In contrast, our application of group-based IOPP is more pervasive; we use it in a two-tiered setting leading to group polynomials of size $O(\sqrt{n})$, where one of the group oracles is adversarially generated. We additionally optimise the group sumcheck to use MSMs instead of group-exponentiations. While both [25, 50] asymptotically achieve $\text{polylog}(n)$ proof size and verification, we employ only one round of folding to *square-root* size in the IOPP to terminate at the instance size of $O(\sqrt{n})$. Concretely, this results in better proof sizes as our initial oracles are small ($\propto \sqrt{n}$) and the overhead of multiple rounds of Merkle tree openings to achieve $\text{polylog}(n)$ efficiency exceeds sending the $O(\sqrt{n})$ sized instance in full. Smaller Merkle trees also increase the effectiveness of path compression, with nodes near the top shared amongst many paths – a property leveraged by our implementation.

3 Preliminaries

3.1 Notation

For $n \in \mathbb{N}$, we denote the sequence of integers $(1, \dots, n)$ by $[n]$. We denote the binary representation of an integer $i \in \mathbb{N}$ by $\langle i \rangle$. We denote λ to be a security parameter. We use negl to denote a negligible function: for any integer $c > 0$, there exists a $n \in \mathbb{N}$ such that for all $x > n$, $\text{negl}(x) \leq 1/x^c$. Throughout, $\mathbb{F}$ denotes a field of prime order p and $\mathbb{G}$ denotes a group of prime order p . We use $L \subseteq \mathbb{F}$ to denote a *smooth* subset of $\mathbb{F}$ i.e. it is a multiplicative coset of $\mathbb{F}^*$ whose order is a power of 2. For $k \in \mathbb{N}$, we define $L^{(k)} := \{x^k : x \in L\}$. We use $\mathbb{F}^{\leq d}[X_1, \dots, X_m]$ to denote the set of all m -variate multivariate polynomials over $\mathbb{F}$ of individual degree at most d . In particular, $\mathbb{F}^{\leq d}[X]$ denotes the set of univariate polynomials of degree at most d , and $\mathbb{F}^{\leq 1}[X_1, \dots, X_m]$ denotes the set of m -variate multilinear polynomials (of size $(n := 2^n)$). We reserve the “tilde” symbol over a function (e.g $\tilde{f}$) to indicate that it is a *multilinear* polynomial. Similarly, a “hat” symbol over a function (e.g $\hat{f}$) denotes the associated univariate polynomial of the multilinear polynomial $\tilde{f}$. The oracle symbol $\llbracket \tilde{f} \rrbracket$ denotes the oracle access to the Reed Solomon encoding of $\tilde{f}$ over a smooth domain. We define the equality polynomial $\tilde{eq}$ to be the multilinear extension of the equality function $eq : \mathbb{F}^{2m} \rightarrow \mathbb{F}$

$$\tilde{eq}(X, Y) = \prod_{i=1}^m (X_i \cdot Y_i + (1 - X_i) \cdot (1 - Y_i)).$$

For two functions $f, g : L \rightarrow \mathbb{F}$, $\Delta(f, g)$ denotes the relative Hamming distance between f and g . For a set $S \subseteq \mathbb{F}^{|L|}$, $\Delta(f, S) := \min_{h \in S} \Delta(f, h)$.

⁴with the caveat that the implementation of Spartan is zero-knowledge, unlike ours

3.2 Interactive Oracle Proofs

Definition 3.1. Interactive Oracle Proofs(IOPs)[14, 43] are information theoretic objects that generalize the notion of Probabilistically Checkable Proofs(PCPs)[6, 7, 10, 26]. An IOP is an interactive proof where the verifier $\mathcal{V}$ is given query access to prover $\mathcal{P}$'s message at each round. An IOP is an interactive proof for a relation $\mathcal{R} = (x, w)$ where $\mathcal{R}$ is an oracle relation such the statement x consists of oracles to polynomials over a field $\mathbb{F}$. We denote $\langle \mathcal{P}, \mathcal{V} \rangle(x)$ the output of $\mathcal{V}$ after interacting with $\mathcal{P}$ on input x ; which is accept or reject.

3.3 Interactive Oracle Proofs of Proximity

An IOP of proximity is for a family of codes C where both prover $\mathcal{P}$ and the verifier $\mathcal{V}$ receive as common input a specification of a code $C \in C$ which is a set of functions $f : L \rightarrow \mathbb{F}$ for $L \subseteq \mathbb{F}$. We also assume that $\mathcal{V}$ has oracle access to a function $f^{(0)} : L \rightarrow \mathbb{F}$ and that $\mathcal{P}$ also receives the same function as the explicit input.

Definition 3.2. An r -round IOPP $S = (\mathcal{P}, \mathcal{V})$ is a $(r + 1)$ -round IOP for the error correcting code $C = \{f : L \rightarrow \mathbb{F}\}$ with respect to a distance measure Δ if

- The first prover message denoted by $\llbracket f^{(0)} \rrbracket$, is a purported codeword of C
- $\Pr[\langle \mathcal{P}, \mathcal{V} \rangle(\llbracket f^{(0)} \rrbracket) = \text{accept} | \Delta(f^{(0)}, C) = 0] = 1$ and
- For any malicious prover $\mathcal{P}^*$, $\Pr[\langle \mathcal{P}^*, \mathcal{V} \rangle = \text{reject} | \Delta(f^{(0)}, C) = \delta] \leq \text{negl}$

The sum of lengths of all prover message except for the first message is the IOPP proof length, total number of queries is the query complexity.

Our construction leverages interactive oracle proofs of proximity (IOPPs) as defined above.

3.4 Polynomial Commitments from IOPPs

Several efficient constructions of univariate polynomial commitment schemes [4, 5, 15, 36, 49] and associated SNARKs [1, 11–13] in the transparent setting have leveraged IOPPs which allow a verifier to succinctly verify that a committed function is close to some linear code. Recent works [5, 49] have proposed IOPPs for multilinear polynomials. In this work, we adapt techniques from multilinear IOPPs to groups in designing the polynomial commitment scheme Titan. Similar to [49], we describe our polynomial commitment scheme (PCS) in the setting of interactive oracle proofs, where oracle to the function is treated as the commitment. Additionally, we consider *extractability* under random point evaluation, which allows extracting the polynomial from an adversary who succeeds in proving evaluation on uniformly sampled points. This restricted notion of extractability suffices for several applications of the PCS, and can be easily extended to standard extractability in the case of multilinear polynomials, using an additional sum-check.

Definition 3.3. A polynomial commitment scheme PC for multilinear polynomials over field $\mathbb{F}$ consists of a tuple of oracle algorithms (**Setup**, **Commit**, **Open**, **Eval**) such that

- **Setup**($1^\lambda, m$) $\rightarrow$ pp takes the security parameter λ and the number of variables $m \in \mathbb{N}$ and outputs public parameters pp.

- **Commit**(pp, $\tilde{f}$) $\rightarrow \llbracket f \rrbracket$ takes public parameters pp and the multilinear polynomial $\tilde{f} \in \mathbb{F}^{\leq 1}[X_1, \dots, X_m]$ and outputs a function oracle $\llbracket f \rrbracket$ (analogue of commitment in standard model).
- **Open**(pp, $f, \tilde{f}$) $\rightarrow b$ is a relation that takes public parameters pp, a function (word) f , a multilinear polynomial $\tilde{f}$, and outputs a bit b indicating accept or reject.
- **Eval**(pp, $\llbracket f \rrbracket, m, z, \sigma; \tilde{f}$) $\rightarrow b$ is an interactive oracle protocol $\langle \mathcal{P}_{\text{Eval}}(f), \mathcal{V}_{\text{Eval}} \rangle(\text{pp}, \llbracket f \rrbracket, m, z, \sigma)$ between a PPT prover and a PPT verifier with common inputs consisting of $\llbracket f \rrbracket$, an evaluation point $z \in \mathbb{F}^m$ and a value $\sigma \in \mathbb{F}$. $\mathcal{P}$ additionally knows a multilinear polynomial $\tilde{f}$. $\mathcal{P}$ wants to convince $\mathcal{V}$ that $\tilde{f}$ is an opening to $\llbracket f \rrbracket$ and $\tilde{f}(z) = \sigma$. $\mathcal{V}$ outputs a bit b at the end of the protocol indicating accept or reject.

PC must satisfy completeness, binding and knowledge soundness.

- PC is **complete** if for any $\lambda, m \in \mathbb{N}$, $\tilde{f} \in \mathbb{F}^{\leq 1}[X_1, \dots, X_m]$ and $z \in \mathbb{F}^m$

$$\Pr \left[\text{Eval}(\text{pp}, \llbracket f \rrbracket, m, z, \sigma; \tilde{f}) = 1 \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda, m); \\ \llbracket f \rrbracket \leftarrow \text{Commit}(\text{pp}; \tilde{f}); \\ \sigma \leftarrow \tilde{f}(z) \end{array} \right] = 1$$

- PC is **binding** if for any $m \in \mathbb{N}$ and PPT adversary $\mathcal{A}$,

$$\Pr \left[b_0 = b_1 = 1 \wedge \tilde{f}_0 \neq \tilde{f}_1 \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda, m); \\ (\llbracket f \rrbracket, \tilde{f}_0, \tilde{f}_1) \leftarrow \mathcal{A}(\text{pp}); \\ b_0 \leftarrow \text{Open}(\text{pp}; \llbracket f \rrbracket, \tilde{f}_0); \\ b_1 \leftarrow \text{Open}(\text{pp}; \llbracket f \rrbracket, \tilde{f}_1) \end{array} \right] \leq \text{negl}(\lambda)$$

- PC is **knowledge sound with random point evaluation** if for any PPT adversary $\mathcal{A}$, there exists a PPT algorithm Ext such that the following probability is negligible in λ :

$$\Pr \left[\begin{array}{l} b = 1 \wedge \\ \mathcal{R}_{\text{Ev}}(\text{pp}, \llbracket f \rrbracket, z, \sigma; \tilde{f}) = 0 \end{array} \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda, m), \\ \llbracket f \rrbracket \leftarrow \mathcal{A}(\text{pp}), \\ z \leftarrow \mathbb{F}^m, \sigma \leftarrow \mathcal{A}(\text{pp}, z), \\ \tilde{f} \leftarrow \text{Ext}^{\mathcal{A}}(\text{pp}, \llbracket f \rrbracket) \\ b \leftarrow \langle \mathcal{A}, \mathcal{V}_{\text{Ev}} \rangle(\text{pp}, \llbracket f \rrbracket, z, \sigma) = 1 \end{array} \right]$$

where the oracle relation $\mathcal{R}_{\text{Ev}}$ is defined as

$$\mathcal{R}_{\text{Ev}} = \{((\text{pp}, \llbracket f \rrbracket, z, \sigma); \tilde{f}) : \text{Open}(\text{pp}, \tilde{f}; \tilde{f}) = 1 \wedge \tilde{f}(z) = \sigma\}.$$

3.5 IOPP for Multilinear Polynomials

We briefly summarize tools we use from multilinear IOPPs such as [5, 49]. Our definitions and notation borrow heavily from WHIR [5]. In particular, we consider IOPPs for *Constrained Reed Solomon Codes* for polynomial evaluation claims. Let $m \in \mathbb{N}$ be a parameter and let $n = 2^m$. We consider multilinear polynomials $\tilde{f} \in \mathbb{F}^{\leq 1}[X_1, \dots, X_m]$, and evaluation claims of the type $\tilde{f}(z) = \sigma$ for some $z \in \mathbb{F}^m$ and $\sigma \in \mathbb{F}$. In WHIR, the multilinear polynomial $\tilde{f}$ is encoded as a Reed-Solomon codeword over a suitably defined domain.

Definition 3.4 (Reed Solomon Code). For finite field $\mathbb{F}$, a domain $L \subseteq \mathbb{F}$ and $m \in \mathbb{N}$, we define *Reed Solomon Code* $C := \text{RS}[\mathbb{F}, L, m]$ as the set of functions $f : L \rightarrow \mathbb{F}$ for which there exists a polynomial $p \in \mathbb{F}[X]$ of degree $< 2^m$ such that $f(x) = p(x)$ for all $x \in L$.

We assume that $\rho := 2^m/|L|$ where $\rho \in (0, 1)$, as the rate of C . To map multilinear polynomial $\tilde{f}$ to a Reed-Solomon codeword, we define the associated univariate polynomial

$$\widehat{f}(X) := \tilde{f}(X, X^2, \dots, X^{2^{m-1}}) \quad (4)$$

which has degree $< 2^m$. Then, we say that the codeword $(\widehat{f}(x))_{x \in L}$ encodes the multilinear polynomial $\tilde{f}$ as a codeword in $\text{RS}[\mathbb{F}, L, m]$. Going ahead, we continue to use the notation $\widehat{f}$ to denote the univariate polynomial of degree $< 2^m$ associated with m -variate multilinear polynomial $\tilde{f}$ given by Equation (4).

To denote the multilinear polynomials satisfying certain evaluation claims, WHIR introduces *constrained* Reed-Solomon codes. Specifically, given an $(m+1)$ -variate multilinear polynomial $\tilde{w}(Z, X)$ with X denoting variables $X_1, \dots, X_m$ and $\sigma \in \mathbb{F}$, we define constrained Reed-Solomon code as the subcode of $\text{RS}[\mathbb{F}, L, m]$ as:

Definition 3.5 (Constrained RS code [5]). For $\mathbb{F}, L, m, \tilde{w}, \sigma$ as defined above. A *constrained Reed-Solomon code* $\text{CRS}[\mathbb{F}, L, m, \tilde{w}, \sigma]$ is defined as the set of codewords in $\text{RS}[\mathbb{F}, L, m]$, consisting of evaluations of polynomials $\widehat{f} \in \mathbb{F}^{< 2^m}[X]$ over L such that their equivalent multilinear polynomial $\tilde{f} \in \mathbb{F}[X_1, \dots, X_m]$ satisfies $\sum_{b \in \{0,1\}^m} \tilde{w}(\tilde{f}(b), b) = \sigma$.

In particular, taking $\tilde{w}(Z, X) = Z \cdot \tilde{e}q(X, \alpha)$ in the above definition, the subcode consists of codewords corresponding to polynomials $\tilde{f}$ such that $\tilde{f}(\alpha) = \sigma$. As a key ingredient, WHIR [5], presents IOPP for constrained Reed-Solomon codes. The IOPPs in [5, 49] proceed in several iterations; each iteration involves k “sum-check” rounds to reduce the instance of proximity testing with respect to code of dimension 2^m to checking proximity with respect to a code of dimension 2^{m-k} . We use simplified variants of above IOPPs, where we restrict to single iteration with $k \approx m/2$. Using two-tiered commitments ensures that the group oracles are small $\approx \sqrt{n}$ – with one iteration offering the best trade-off for practical values of n . Therefore, a single iteration of WHIR, adapted to groups is a key tool we use in our construction. We give a self-contained description of an iteration in WHIR in the Appendix in Section C.

We next discuss folding of functions, a key notion used in IOPPs [5, 11, 49]. Throughout the paper, we consider functions over *smooth domains*, i.e, subgroups of roots of unity of order 2^r for some $r \in \mathbb{N}$.

Definition 3.6 (Folded Functions). Let $L \subseteq \mathbb{F}$ be a smooth domain and $f : L \rightarrow \mathbb{F}$ be a function. Let $L^{(2)}$ denote the set $\{x^2 : x \in L\}$, i.e, the set of “squares” in L . For $\alpha \in \mathbb{F}$, we define the function $f_\alpha := \text{fold}(f, \alpha)$ as the function $f_\alpha : L^{(2)} \rightarrow \mathbb{F}$, with $f_\alpha(x^2)$ for $x \in L$ defined by:

$$f_\alpha(x^2) = \frac{f(x) + f(-x)}{2} + \alpha \cdot \frac{f(x) - f(-x)}{2x} \quad (5)$$

Recall that we use $L^{(t)}$ to denote the set $\{x^t : t \in L\}$.

The above definition also implies that given oracle access to f over L , oracle access to f_α over $L^{(2)}$ can be simulated via two oracle queries to f at the square-roots of the queried point. We can extend Definition 3.6 to describe *multi-folding* of a function using $\alpha = (\alpha_1, \dots, \alpha_k)$ as:

$$\text{fold}(f, (\alpha_1, \dots, \alpha_k)) = \text{fold}(\text{fold}(f, \alpha_1, \dots, \alpha_{k-1}), \alpha_k)$$

We note that in this case, the function $\text{fold}(f, \alpha)$ is defined over the domain $L^{(2^k)}$. It is also easy to verify that oracle access to $\text{fold}(f, \alpha)$ can be simulated using 2^k oracle queries to f at the 2^k pre-images of the queried point under the map $x \mapsto x^{2^k}$. We use this property to realize efficient oracle access to f . The folding of “polynomial” functions is nicely captured by the following Lemma.

LEMMA 3.7 ([5]). For a function $f : L \rightarrow \mathbb{F}$ given by $f(x) = \tilde{f}(x)$ for $\tilde{f} \in \mathbb{F}[X_1, \dots, X_m]$ and $\alpha \in \mathbb{F}^k$ for some $k \leq m$, we have $\text{fold}(f, \alpha)(y) \equiv \tilde{f}(\alpha, y)$ for $y \in L^{(2^k)}$. Here x and y denote the vectors $(x, x^2, \dots, x^{2^{m-1}})$ and $(y, y^2, \dots, y^{2^{m-k-1}})$ respectively.

3.5.1 Encoding Multilinear Polynomial Oracles. Multilinear IOPPs (e.g. [5]) require queries to the folded function $\text{fold}(f, \alpha)$. We earlier noted that a query to $\text{fold}(f, \alpha)$ can be simulated using 2^k queries to f . We now provide a more explicit description of the same. For the honest prover, we have $f = \tilde{f}$ for some m -variate multilinear polynomial. We note that for $y = x^{2^k} \in L^{(2^k)}$, we have:

$$\begin{aligned} \text{fold}(f, \alpha)(y) &= \tilde{f}_\alpha(y) = \tilde{f}(\alpha, y, y^2, \dots, y^{2^{m-k-1}}) \quad (\text{Lemma 3.7}) \\ &= \sum_{b \in \{0,1\}^k} \tilde{e}_q(\alpha, b) \cdot \tilde{f}(b, y, y^2, \dots, y^{2^{m-k-1}}) \end{aligned} \quad (6)$$

Thus, we commit to f as follows: For each $y \in L^{(2^k)}$, the prover commits to the vector

$$(f(b, y, y^2, \dots, y^{2^{m-k-1}}))_{b \in \{0,1\}^k}$$

as one leaf of the Merkle tree. Thus, each leaf of the Merkle tree corresponds evaluations of multilinear polynomial $\tilde{p}_y(b) = \tilde{f}(b, y, \dots, y^{2^{m-k-1}})$ over $b \in \{0,1\}^k$. It can be shown that $\tilde{p}_y$ is completely determined by y and evaluations of $\tilde{f}$ on the coset $\{x : x^{2^k} = y\}$, and thus the above commitment can be seen as committing to f over L in a coset-wise manner. To open $\text{fold}(f, \alpha)(y)$, the prover opens the above vector and sends it to the verifier, who checks consistency with the Merkle root and computes $\text{fold}(f, \alpha)(y)$ using Equation (6). Thus, the verifier effort to verify q openings is $O(q(m + 2^k))$, and the prover needs to provide only q authentication paths instead of $q \cdot 2^k$ paths.

3.6 Reed-Solomon Codes over Groups

We now define Reed-Solomon codes over group $\mathbb{G}$ of prime order p . Let $L \subseteq \mathbb{F}$ be a domain and $m \in \mathbb{N}$. We define $\text{RS}[\mathbb{G}, L, m]$ as the set of functions $g : L \rightarrow \mathbb{G}$ such that there exists a polynomial $\tilde{g}(X) \in \mathbb{G}[X]$ of degree $< 2^m$ such that $g(x) = \tilde{g}(x)$ for all $x \in L$. We note that the codes $\text{RS}[\mathbb{F}, L, m]$ and $\text{RS}[\mathbb{G}, L, m]$ have identical generator matrices, and hence the same distance properties. This follows from the (vector space) isomorphism $x \mapsto x \cdot g$ between $\mathbb{F}$ and $\mathbb{G}$ for any generator g of $\mathbb{G}$. In subsequent sections, use WHIR IOPP for Reed-Solomon codes over $\mathbb{G}$.

In the extended preliminaries Section C in Appendix, we also describe generalization of sumcheck protocol to polynomials over groups.

4 Titan: Overview

We now expand on the construction of Titan, our transparent polynomial commitment scheme sketched in Section 2. Let $\mathbb{G}$ denote a subgroup of prime order in an elliptic curve E , where the discrete-log problem is hard. We use $\mathbb{F}$ to denote the scalar field $\mathbb{Z}_p$. We also assume $\mathcal{H}$ to be a collision resistant hash function $\mathcal{H} : \mathbb{G}^* \rightarrow \{0,1\}^{2\lambda}$. As seen earlier, the core technique in Titan is to decompose the evaluation claim into an evaluation claim over a group oracle, and a proof of knowledge using CSP. We set both

Polynomial-size parameters	
m	Number of variables; $n = 2^m$ is the polynomial size
p, q	Sizes of the two families of group polynomials; $n = p \cdot q$
$\ell_p = \log p$	Number of variables in the outer ($\tilde{G}$) group polynomial
$\ell_q = \log q$	Number of variables in the inner/generator ($\tilde{g}$) group polynomial
$g = (g_1, \dots, g_q)$	Uniformly sampled generators from $\mathbb{G}$ (commitment key)
Reed-Solomon encoding parameters	
$\rho_1, \rho_2 \in (0, 1)$	Code rates for the outer and inner RS codes
$L_1 \subseteq \mathbb{F}$	Smooth multiplicative domain of size $\rho_1^{-1} \cdot p$ (outer)
$L_2 \subseteq \mathbb{F}$	Smooth multiplicative domain of size $\rho_2^{-1} \cdot q$ (inner)
Folding / coset parameters	
k_1, k_2	Folding factors for outer and inner codes
ℓ_1, ℓ_2	Coset size (folding arity) for code $i \in \{1, 2\}$
$m_1 = p/\ell_1$	Size of folded outer polynomial
$m_2 = q/\ell_2$	Size of folded inner polynomial
$K_i = L_i^{(\ell_i)}$	Folded sub-domain (ℓ_i -th powers of L_i)
ℓ_1, ℓ_2	Number of consistency check queries in outer and inner group IOPPs
Oracles	
$\llbracket g \rrbracket$	WHIR oracle committing to generator polynomial $\tilde{g}$; locally held by $\mathcal{V}$

Figure 1: Parameter summary for the Titan PCS

instance sizes to be $\approx O(\sqrt{n})$. In this section, we fully specify parameters, which can be optimised for specific settings.

4.1 Setup

Let m denote the number of variables and $n = 2^m$ denote the size of the polynomials. Let $g = (g_1, \dots, g_q)$ be a vector of uniformly and independently sampled generators from $\mathbb{G}$. We consider group polynomials of two sizes - polynomials of size p obtained as commitment of the field polynomial and polynomial of size q as the MLE of generators g . Observe that we have $n = p \cdot q$. We refer to former as the *outer* group polynomial, while to the latter as *inner* group polynomial. Next, we choose parameters to encode the group polynomials as Reed-Solomon codes. Let $\rho_1, \rho_2 \in (0, 1)$ denote the rate of the Reed-Solomon codes used to encode the group polynomials. Accordingly, let L_1, L_2 be smooth multiplicative subgroups of $\mathbb{F}$ of sizes $\rho_1^{-1} \cdot p$ and $\rho_2^{-1} \cdot q$ respectively. Let $\ell_p = \log p$ and $\ell_q = \log q$ denote the number of variables outer and inner group polynomials respectively. We also define sub-domains K_1 and K_2 of L_1 and L_2 respectively to encode folded functions. For folding factors k_1 and k_2 , we set $\ell_i = 2^{k_i}$, $i \in [2]$ and define $K_i = L_i^{(\ell_i)}$.

Encoding of Generators. The verifier also computes one-time encoding of the polynomial $\tilde{g}(X)$ interpolating the generators g . Following the coset-wise encoding in Section 3.5.1, the oracle $\llbracket g \rrbracket$ when queried on $y \in K_2$ returns the evaluations $(\tilde{g}(b, y))_{b \in \{0,1\}^{k_2}}$, where $y = (y, y^2, \dots, y^{2^{\ell_q - k_2 - 1}}) \in \mathbb{F}^{\ell_q - k_2}$. We set $\ell_2 = O(\sqrt{q})$ and thus $|K_2| = O(\sqrt{q})$. When we set $q = O(\sqrt{n})$, both ℓ_2 and $|K_2|$ are $O(\sqrt{n})$. We summarize the parameters in Figure 1 for quick reference.

4.2 Commitment

Let $\tilde{f} \in \mathbb{F}^{\leq 1}[X_1, \dots, X_m]$ be a multilinear polynomial. Like prior work such as Hyrax [47] and Dory [38], we view $\tilde{f}$ as $p \times q$ matrix,

with entries $f_{ij} := \tilde{f}(\langle i \rangle, \langle j \rangle)$.

$$\begin{array}{cccc} g_1 & g_2 & \cdots & g_q \\ \begin{pmatrix} f_{11} & f_{12} & \cdots & f_{1q} \\ f_{21} & f_{22} & \cdots & f_{2q} \\ \vdots & \vdots & \ddots & \vdots \\ f_{p1} & f_{p2} & \cdots & f_{pq} \end{pmatrix} & G_1 & G_2 & \vdots & G_p \end{array}$$

The above matrix view is captured by the following decomposition:

$$\tilde{f}(\mathbf{X}, \mathbf{Y}) = \sum_{i=1}^p \tilde{eq}(\mathbf{X}, \langle i \rangle) \cdot \tilde{f}(\langle i \rangle, \mathbf{Y}) \quad (7)$$

Here $\mathbf{X}$ and $\mathbf{Y}$ denote blocks of variables $(X_1, \dots, X_{\ell_p})$ and $(Y_1, \dots, Y_{\ell_q})$ respectively, while $\langle i \rangle$ denotes the point in $\{0, 1\}^{\ell_p}$ canonically determined by i . Using $\mathbf{g}$ as a commitment key, we define the homomorphic commitment to polynomial $\tilde{h} \in \mathbb{F}^{\leq 1}[X_1, \dots, X_{\ell_q}]$ as:

$$\text{Com}(\tilde{h}; \mathbf{g}) = \sum_{i=1}^q \tilde{h}(\langle i \rangle) \cdot g_i \quad (8)$$

Now, let $\tilde{f}_i(\mathbf{Y}) := \tilde{f}(\langle i \rangle, \mathbf{Y})$ denote the i^{th} polynomial in the decomposition of $\tilde{f}$ in Equation (7). Let $G_i = \text{Com}(\tilde{f}_i; \mathbf{g})$ be the commitment to $\tilde{f}_i$ for $i \in [p]$. At this point, we compute an *outer commitment* over the vector $(G_1, \dots, G_p)$, which commits to $\tilde{f}$. Instead of relying on bilinear pairings, we interpolate a group polynomial $\tilde{G}(\mathbf{X})$ such that $\tilde{G}(\langle i \rangle) = G_i$ for $i \in [p]$ and commit to $\tilde{G}$ as an oracle $\llbracket G \rrbracket$. Formally, we have:

$$\tilde{G}(\mathbf{X}) = \sum_{i=1}^p \tilde{eq}(\mathbf{X}, \langle i \rangle) \cdot G_i \quad (9)$$

Next, we specify the encoding of $\tilde{G}$ as an oracle.

Encoding of $\tilde{G}$. As in Section 3.5.1, we commit to $\tilde{G}$ as evaluations over cosets for $y \in K_1$. For $y \in K_1$, let $\mathbf{y}$ denote the vector $(y, y^2, \dots, y^{2^{\ell_p - k_1 - 1}}) \in \mathbb{F}^{\ell_p - k_1 - 1}$. Then for each $y \in K_1$, the oracle $\llbracket G \rrbracket$ returns the vector $(\tilde{G}(\mathbf{b}, \mathbf{y}))_{\mathbf{b} \in \{0,1\}^{k_1}}$. To realize the oracle $\llbracket G \rrbracket$, we commit G as a Merkle tree with $m_1 := |K_1|$ leaves, each being a vector of size 2^{k_1} . For our construction, we set $\ell_1, m_1 \approx O(\sqrt{p})$. When $p = O(\sqrt{n})$, we have $\ell_1, m_1 = O(\sqrt{n})$.

4.3 Evaluation Protocol for Titan PCS

In the evaluation PIOP, the verifier $\mathcal{V}(\mathbf{z}, \sigma, \llbracket G \rrbracket)$ with inputs $(\mathbf{z}, \sigma)$ and oracle access $\llbracket G \rrbracket$ needs to be convinced that $\llbracket G \rrbracket$ encodes a multilinear polynomial $\tilde{G}$, which in turn commits to the field polynomial $\tilde{f} \in \mathbb{F}[X_1, \dots, X_m]$ satisfying $\tilde{f}(\mathbf{z}) = \sigma$. Let us parse $\mathbf{z} = (\mathbf{x}, \mathbf{y})$ with $\mathbf{x} \in \mathbb{F}^{\ell_p}$ and $\mathbf{y} \in \mathbb{F}^{\ell_q}$. Since $\tilde{G}$ commits to $\tilde{f}$ row wise, by homomorphism of the Pedersen commitment, $\tilde{G}(\mathbf{x})$ is the Pedersen commitment of the polynomial $\tilde{f}(\mathbf{x}, \mathbf{Y})$. Therefore, we have the following meta-protocol for evaluation:

- (1) $\mathcal{P}$ sends commitment C (purportedly to $\tilde{f}(\mathbf{x}, \mathbf{Y})$).
- (2) $\mathcal{P}$ and $\mathcal{V}$ execute WHIR IOPP to check that $\tilde{G}(\mathbf{x}) = C$, for $\tilde{G}$ encoded by $\llbracket G \rrbracket$.

- (3) $\mathcal{P}$ shows knowledge of field polynomial with coefficients $\mathbf{a} \in \mathbb{F}^q$ via the *proof of knowledge* protocol:

$$\text{PoK} \left\{ (C, \sigma, \mathbf{g}; \mathbf{a}) : \sum_{i=1}^q a_i \cdot g_i = C \wedge \sum_{i=1}^q a_i \cdot \tilde{eq}(\mathbf{y}, \langle i \rangle) = \sigma \right\}$$

Decoding G to a polynomial: More precisely, the second step of the meta-protocol involves choosing a proximity parameter $\delta \in (0, 1)$ followed by $\mathcal{P}$ and $\mathcal{V}$ invoking WHIR IOPP to check that G is δ -close to $\text{CRS}[\mathbb{G}, L_1, \ell_p, \tilde{w}, \sigma]$, where $\tilde{w}(Z, \mathbf{X}) = Z \cdot \tilde{eq}(\mathbf{X}, \mathbf{x})$. We choose an IOPP involving *single* WHIR iteration (detailed in Appendix C.1), which concretely leads to better proof sizes (our oracles are $O(\sqrt{n})$ to start with). Further, note that an IOPP only ensures that $\llbracket G \rrbracket$ is close to “some” polynomial satisfying the evaluation claim. For a polynomial commitment scheme we need to bind G to a unique polynomial, for which there are two popular approaches:

- *Unique Decoding Regime:* In this regime, the proximity parameter δ is set below the unique decoding radius of the code, i.e., $\delta < (1 - \rho_1)/2$, which ensures there is a unique polynomial δ -close to G and hence all evaluation claims hold with respect to this polynomial. This regime incurs larger proof sizes due to higher query complexity to achieve target soundness.
- *List Decoding Regime:* In this regime, we use proximity parameter beyond the unique decoding radius all the way up to $1 - \sqrt{\rho_1} - \gamma$ (Johnson’s radius) for some $\gamma > 0$. Using the standard technique of *out of domain* sample, the verifier can bind prover to a unique polynomial in the δ -ball for evaluation proofs.

We describe and prove the soundness of Titan PCS in the unique decoding regime, choosing to focus more on the interplay of group based IOPPs with compressed sigma protocol to achieve overall soundness (and knowledge-soundness). The construction and security proofs can be adapted to list-decoding regime in a standard manner which we sketch in Section 5.4. Our implementation and benchmarks are with respect to proximity parameters under list-decoding regime.

5 Titan PCS: Complete Protocol

We elaborate on the protocols for two checks required by $\mathcal{V}$ in the final two steps of the meta-protocol in Section 4.3.

5.1 Outer Group Polynomial Evaluation

In Figure 2, we present an adaptation of WHIR iteration to group polynomials to check G is δ -close to $\text{CRS}[\mathbb{G}, L_1, \ell_p, \tilde{w}, C]$ given $\delta \in (0, 1)$, $\tilde{w}(Z, \mathbf{X}) = Z \cdot \tilde{eq}(\mathbf{X}, \mathbf{x})$ for $\mathbf{x} \in \mathbb{F}^{\ell_p}$ and $C \in \mathbb{G}$. We have the following:

LEMMA 5.1. *For $\delta \in (0, 1 - \sqrt{\rho_1})$, query bound t and other parameters as in Figure 1, the protocol in Figure 2 accepts with probability 1 whenever $G \in \text{CRS}[\mathbb{G}, L_1, \ell_p, \tilde{w}, C]$. For G which is δ -far from $\text{CRS}[\mathbb{G}, L_1, \ell_p, \tilde{w}, C]$ it rejects e.w.p. $\approx (1 - \delta)^t$. The prover incurs MSM of size $O(p)$ in $\mathbb{G}$, while $\mathcal{V}$ incurs MSM of size $m_1 + t \cdot \ell_1$ and $t \cdot (m_1 + \ell_1)$ $\mathbb{F}$ operations. The communication is $3k_1 + m_1 + t\ell_1$ $\mathbb{G}$ elements.*

PROOF. The completeness of the protocol can be routinely verified. We show soundness. Assume that an adversarial prover $\mathcal{P}^*$

- **Public Parameters:** As in Figure 1.
- **Common Input:** $(x, C, \delta, \llbracket G \rrbracket)$.
- **Prover's Input:** $\tilde{G} \in \mathbb{G}^{\leq 1}[X_1, \dots, X_{\ell_p}]$.
- **Protocol (Π_{outer}):**
 - (1) In round i for $i \in [k]$ (for convenience $k := k_1, m := m_1$ in Figure 1):
 - $\mathcal{P}$ computes $h_i(X) = \sum_{b \in \{0,1\}^{m-i}} \tilde{w}(\tilde{G}(\alpha_i, X, b), \alpha_i, X, b)$, where $\alpha_i = (\alpha_{i,1}, \dots, \alpha_{i,i-1})$ denotes the vector of challenges sent in the first $i-1$ rounds.
 - $\mathcal{P}$ sends $h_i(X)$.
 - $\mathcal{V}$ checks: If $i > 1$, $\mathcal{V}$ checks $h_i(0) + h_i(1) = h_{i-1}(\alpha_{i-1})$. For $i = 1$, it checks $h_1(0) + h_1(1) = C$.
 - $\mathcal{V}$ sends $\alpha_i \leftarrow \mathbb{F}$.
 - (2) $\mathcal{P}$ sends evaluations $(c_1, \dots, c_m)$ of the polynomial $\tilde{G}'(\cdot) = \tilde{G}(\alpha, \cdot)$, where $\alpha = \alpha_{k+1}$.
 - (3) $\mathcal{V}$ samples t random points $z_1, \dots, z_t \leftarrow K_1$.
 - (4) $\mathcal{V}$ obtains $v_i \in \mathbb{G}^{\ell_i}$ by querying $\llbracket G \rrbracket$ at z_i for all $i \in [t]$.
 - (5) $\mathcal{V}$ checks consistency of $\tilde{G}'$ with $\llbracket G \rrbracket$: $\forall i \in [t]$
 - Check $\sum_{j=1}^m c_j \cdot \tilde{e}q(\langle j \rangle, z_i) = \sum_{j=1}^m v_i[j] \cdot \tilde{e}q(\langle j \rangle, x)$ where $z_i = (z_i, z_i^2, \dots, z_i^{2^{p-1}})$.
 - (6) $\mathcal{V}$ checks reduced sum-check claim:
 - Check: $\sum_{j=1}^m c_j \cdot \tilde{e}q(\alpha, \langle j \rangle, x) = h_k(\alpha_k)$.
 - (7) $\mathcal{V}$ accepts if all the checks pass, else it rejects.

Figure 2: IOPP for Outer Group Polynomial Evaluation

produces a transcript

$$(G, h_1, \alpha_1, \dots, h_k, \alpha_k, \tilde{G}').$$

We show that $\mathcal{V}$ rejects the transcript if $\Delta(G, \tilde{G}) > \delta$ for all $\tilde{G}$ satisfying $\tilde{G}(x) = C$. Let $\alpha = (\alpha_1, \dots, \alpha_k)$ denote the vector of $\mathcal{V}$'s challenges. If one of the sum-check conditions fail, $\mathcal{V}$ rejects. Assume that polynomials $h_1, \dots, h_k$ satisfy sum-check verification constraints. Then, by Lemma C.1, for $\delta < 1 - \sqrt{\rho}$, if G is δ -far from every polynomial $\tilde{H}$ satisfying $\tilde{H}(x) = C$, with overwhelming probability $\text{fold}(G, \alpha)$ is δ -far from every polynomial $\tilde{H}'$ satisfying $\sum_{i=1}^m \tilde{H}'(\langle i \rangle) \cdot \tilde{e}q(\alpha, \langle i \rangle, x) = h_k(\alpha_k)$. Clearly, in this case either $\tilde{G}'$ fails the reduced sum-check claim in Step (5), or the consistency check succeeds with probability at most $(1-\delta)^t$. This shows that the protocol has soundness error $\approx (1-\delta)^t$. The verification in Figure 2 checks $t+1$ inner products. The verifier uses a random $\gamma \leftarrow \mathbb{F}$ to batch the checks involving group vectors resulting in MSM of size $t \cdot \ell_1 + m_1$ and $t \cdot (m_1 + \ell_1)$ $\mathbb{F}$ operations. The communication complexity can also be easily verified. $\square$

5.2 Evaluation of partial polynomial

To complete the evaluation proof, we need to show that $\tilde{a}(y) = \sigma$ where the polynomial $\tilde{a}$ opens the commitment C sent by the prover in the meta-protocol in Section 4.3. We can formulate this as a Compressed Sigma Protocol [9], where the prover proves knowledge of coefficient vector $(a_1, \dots, a_q)$ such that:

$$\sum_{i=1}^q a_i \cdot g_i = C \quad \wedge \quad \sum_{i=1}^q a_i \cdot \tilde{e}q(\langle i \rangle, y) = \sigma \quad (10)$$

We note that the linear form evaluating to σ is dictated by our interpretation of $(a_1, \dots, a_q)$ as evaluations of $\tilde{a}$ over the boolean hypercube $\{0,1\}^q$. For the folding parameter $k := k_2$ (see Figure 1) the compressed sigma protocol for Equation 10 proceeds in k rounds; in each round the instance is reduced to a smaller instance using verifier's challenges. After k rounds, the prover sends the

witness for the reduced instance in *full*. We present the protocol in Figure 3. The prover sends the folded vectors g_k and a_k after k rounds. In standard CSP the vectors are folded by splitting them in the middle and combining the two parts using the verifier's challenge. In a slight departure, we split them as consisting of *even* and *odd* positions, which is conceptually equivalent. Our split and fold leads to simpler algebraic interpretation: If $\tilde{g}_i(\cdot)$ is a multilinear polynomial which interpolates the round i vector g_i , the multilinear polynomial $\tilde{g}(\alpha, \cdot)$ interpolates the round $i+1$ vector g_{i+1} with α being the folding challenge. After k rounds, the verifier checks correctness of g_k by querying the oracle $\llbracket G \rrbracket$ folded by challenges $\alpha = (\alpha_1, \dots, \alpha_k)$. The folding of the vector describing the linear form is easier to describe. In a CSP, when $(\alpha, 1-\alpha)$ is used to fold the generators, the reverse $(1-\alpha, \alpha)$ is applied to fold the linear form. Thus the linear form u_k is given as evaluations $\tilde{e}q(\bar{\alpha}, \langle j \rangle, y)$ for $j \in [m_2]$ where $\bar{\alpha} = (1-\alpha_1, \dots, 1-\alpha_k)$. The complete protocol appears in Figure 3.

- **Public Parameters:** pp as in Figure 1.
- **Common Input:** pp, $C \in \mathbb{G}$, $\sigma \in \mathbb{F}$.
- **$\mathcal{P}$'s Input:** $a \in \mathbb{F}^q$.
- **Protocol (Π_{inner}):**
 - (1) $\mathcal{P}$ and $\mathcal{V}$ set $g_0 = g$, $a_0 = a$, $u_0 = (\tilde{e}q(\langle j \rangle, y))_{j \in [q]}$.
 - (2) Set $k := k_2$. For $i = 0$ to $k-1$:
 - $\mathcal{P}$ computes g_i^e, g_i^o of length $q/2^{i+1}$ given by $g_i^e[j] = g[2j]$ and $g_i^o[j] = g[2j+1]$ for $j \in [q/2^{i+1}]$. Similarly, it computes vectors a_i^e, a_i^o, u_i^e , and u_i^o .
 - $\mathcal{P}$ computes:
$$L_i = \langle a_i^o, g_i^e \rangle, R_i = \langle a_i^e, g_i^o \rangle, \ell_i = \langle a_i^o, u_i^e \rangle, r_i = \langle a_i^e, u_i^o \rangle$$
 - $\mathcal{P}$ sends L_i, R_i, ℓ_i, r_i .
 - $\mathcal{V}$ sends $\alpha_i \leftarrow \mathbb{F}$.
 - $\mathcal{P}$ and $\mathcal{V}$ update commitment and linear form:
$$C_{i+1} = \alpha_i(1-\alpha_i) \cdot C_i + (1-\alpha_i)^2 \cdot L_i + \alpha_i^2 \cdot R_i,$$

$$\sigma_{i+1} = \alpha_i(1-\alpha_i) \cdot \sigma_i + (1-\alpha_i)^2 \cdot \ell_i + \alpha_i^2 \cdot r_i$$
 - $\mathcal{P}$ folds vectors: $a_{i+1} = \alpha_i a_i^e + (1-\alpha_i) \cdot a_i^o$, $g_{i+1} = (1-\alpha_i) g_i^e + \alpha_i \cdot g_i^o$, $u_{i+1} = (1-\alpha_i) u_i^e + \alpha_i \cdot u_i^o$.
 - (3) After k rounds, $\mathcal{P}$ sends $g_k \in \mathbb{G}^{m_2}$, $a_k \in \mathbb{F}^{m_2}$.
 - (4) $\mathcal{V}$ samples $z_1, \dots, z_t \in K_2$. It queries $v_i := \llbracket \text{fold}(\tilde{g}, \alpha) \rrbracket(z_i)$.
 - (5) $\mathcal{V}$ checks consistency of g_k with $\tilde{g}$: Let $z_i = (z_i, z_i^2, \dots, z_i^{tq-k-1})$, $\alpha = (\alpha_1, \dots, \alpha_k)$, $\bar{\alpha} = (1-\alpha_1, \dots, 1-\alpha_k)$.
 - $\forall i \in [t]$: $\sum_{j=1}^{m_2} g_k[j] \cdot \tilde{e}q(\langle j \rangle, z_i) \stackrel{?}{=} \sum_{j=1}^{m_2} v_i[j] \cdot \tilde{e}q(\langle j \rangle, \alpha)$.
 - $\mathcal{V}$ checks reduced instance:
$$\sum_{j=1}^{m_2} g_k[j] \cdot a_k[j] \stackrel{?}{=} C_k, \quad \sum_{j=1}^{m_2} a_k[j] \cdot \tilde{e}q(\langle j \rangle, \bar{\alpha}, y) \stackrel{?}{=} \sigma_k$$
 - (6) $\mathcal{V}$ accepts if all the checks succeed.

Figure 3: CSP for partial evaluation

LEMMA 5.2. Let pp denote the public parameters in Figure 1. Let $k := k_2$, $\rho := \rho_2$ and $\mathcal{R}_{\text{inner}}(\text{pp})$ denote the relation consisting of tuples $((C, y, \sigma); a \in \mathbb{F}^q)$ such that $\sum_{i=1}^q a_i \cdot g_i = C$ and $\sum_{i=1}^m a_i \cdot \tilde{e}q(\langle i \rangle, y) = \sigma$. Then, assuming hardness of discrete log in $\mathbb{G}$, the protocol Π_{inner} is complete and $(\mu_1, \dots, \mu_k)$ -special sound for $\mu_i = 3$ for relation $\mathcal{R}_{\text{inner}}$ with soundness error $\varepsilon + 3^k \cdot \rho^t$. Here ε denotes the soundness error of standard CSP for $\mathcal{R}_{\text{inner}}$, where $\mathcal{V}$ also folds the generators.

PROOF. The completeness may be verified straightforwardly. We prove special soundness using the standard tree of accepting transcripts argument. When $\mathcal{V}$ also folds the generators, the protocol in

Figure 3 is an instance of standard compressed sigma protocol [9]. Consider a tree of 3^k accepting transcripts by invoking the CSP tree finding algorithm with rewinding access to adversarial prover $\mathcal{P}^*$. Let $\alpha^{(i)}$ denote the vector of challenges for i^{th} transcript and $\mathbf{g}_k^{(i)}$ denote the vector $\mathbf{g}_k$ in the i^{th} transcript. Let $E_i, i \in [3^k]$ denote the event that $\mathbf{g}_k^{(i)} \neq \tilde{g}(\alpha^{(i)}, \cdot)$, i.e., the vector $\mathbf{g}_k^{(i)}$ does not correspond to correct folding of the generators. If no event E_i occurs, the tree of transcripts is identical to the one in standard CSP, and thus witness can be extracted e.w.p. ϵ . Since we assume $\mathbf{g}$ (which is part of pp) is honestly encoded as a codeword in $\text{RS}[\mathbb{G}, L_2, \log q]$, if $\mathbf{g}_k^{(i)}$ is not identical to coefficients of $\tilde{g}(\alpha, \cdot)$, they determine different codewords in $\text{RS}[\mathbb{G}, K_2, \log m_2]$. It can be seen that the rate of $\text{RS}[\mathbb{G}, K_2, \log m_2]$ is also ρ . Hence, the consistency check in Step (5) failing to catch the discrepancy is at most $(1 - \delta)^t = \rho^t$. Thus, for a fixed i , $\Pr[E_i]$ is upper bound by ρ^t . Thus probability that one of the E_i 's occurs is $\leq 3^k \cdot \rho^t$. This implies a soundness bound of $\epsilon + 3^k \cdot \rho^t$ for the protocol. $\square$

5.3 Complete Protocol

The evaluation protocol for Titan PCS can be now described as a sequential composition of protocols Π_{outer} and Π_{inner} as we describe in Figure 4. We summarise the construction of Titan PCS in Figure 5. The properties are formally captured in Theorem 5.3.

- **Public Parameters:** pp as in Figure 1 and $\delta := (1 - \rho_1)/2$.
- **Common Input:** $\mathbf{z} \in \mathbb{F}^m, \sigma \in \mathbb{F}, \llbracket G \rrbracket$.
- **Prover's Input:** $\tilde{f} \in \mathbb{F}^{\leq 1}[X_1, \dots, X_m], \tilde{G} = \text{Com}(\tilde{f}; \mathbf{g})$.
- **Evaluation Protocol** (Π_{Eval}): Parse $\mathbf{z}$ as $(\mathbf{x}, \mathbf{y})$ with $\mathbf{x} \in \mathbb{F}^{\ell p}$ and $\mathbf{y} \in \mathbb{F}^{\ell q}$.
 - (1) $\mathcal{P}$ sends commitment $C = \text{Com}(\tilde{f}(\mathbf{x}, \mathbf{y}); \mathbf{g})$ to the partial evaluation $\tilde{f}(\mathbf{x}, \mathbf{y})$.
 - (2) $b_{\text{outer}} \leftarrow \langle \mathcal{P}(\text{pp}, \mathbf{x}, C, \delta, \llbracket G \rrbracket); \tilde{G}, \tilde{f}, \mathcal{V}_{\text{outer}}(\text{pp}, \mathbf{x}, C, \delta, \llbracket G \rrbracket) \rangle$
 - (3) $b_{\text{inner}} \leftarrow \langle \mathcal{P}(\text{pp}, C, \mathbf{y}, \sigma; \mathbf{a}), \mathcal{V}_{\text{inner}}(\text{pp}, C, \mathbf{y}, \sigma) \rangle$.
 - (4) $\mathcal{V}$ returns $b_{\text{outer}} \wedge b_{\text{inner}}$

Figure 4: Titan Evaluation Protocol

- **Setup**($1^\lambda, m$): Outputs pp in Figure 1 as described in Section 4.1. Set $\delta := (1 - \rho_1)/2$.
- **Commit**(pp, $\tilde{f}$) $\rightarrow \llbracket G \rrbracket$: Outputs $\llbracket G \rrbracket$ where G is constructed as in Section 4.2.
- **Open**(pp, $\llbracket G \rrbracket; \tilde{f}$) $\rightarrow b$: Computes $\tilde{G} = \text{Com}(\tilde{f}; \mathbf{g})$ as in Section 4.2 and outputs $b = 1$ if $\Delta(\tilde{G}, G) < \delta$. Outputs $b = 0$ otherwise.
- **Eval**: The evaluation protocol $\langle \mathcal{P}_{\text{Eval}}(\tilde{f}), \mathcal{V}_{\text{Eval}} \rangle(\text{pp}, \llbracket G \rrbracket, \mathbf{z}, \sigma)$ is given by Figure 4.

Figure 5: Titan PCS under Unique Decoding Regime

THEOREM 5.3. *The scheme Titan PCS consisting of algorithms (Setup, Commit, Open, Eval) as in Figure 5 for polynomials in $\mathbb{F}^{\leq 1}[X_1, \dots, X_m]$ satisfies completeness, binding, and knowledge soundness with random point evaluation with soundness error $\epsilon_{\text{RPE}} \leq (p+2)(1-\delta)^{t_1} + 3^{k_2} \rho_2^{t_2} + \epsilon_{\text{CSP}} + \epsilon_{\text{DLOG}}$. Here t_1 and t_2 denote number of consistency queries in Π_{outer} and Π_{inner} respectively, while ϵ_{CSP} and ϵ_{DLOG} denote soundness of standard CSP and adversary's success probability in breaking discrete-log in $\mathbb{G}$ respectively.*

Due to space constraints, we defer the detailed proof of Theorem 5.3 to Appendix Section B.

5.4 Extending to List Decoding Regime

We crucially relied on proximity parameter $\delta < (1 - \rho_1)/2$ to bind the prover to a unique polynomial $\tilde{G}$ in the δ -neighborhood of G . For $(1 - \rho_1/2) \leq \delta < 1 - \sqrt{\rho_1} - \gamma$ for a constant $\gamma > 0$, the δ -ball contains up to $\text{poly}(p, \gamma^{-1})$ polynomials in the δ -ball around G . The standard approach to use δ up to $1 - \sqrt{\rho_1}$ (Johnson's radius) is to request evaluation C_0 of the polynomial at a random point $z_0 \leftarrow \mathbb{F}$, called the *out of domain* sample. With overwhelming probability, the claimed evaluation at z_0 uniquely identifies the polynomial in the δ -neighborhood of G . To force the identified polynomial during evaluation, the evaluation claim $\tilde{G}(\mathbf{x}) = C$ is appended with the evaluation claim $\tilde{G}(z_0) = C_0$. When the IOP is compiled to a non-interactive argument in ROM via BCS [14] transformation, the point z_0 can be deterministically obtained by querying the *random oracle* RO with the Merkle tree commitment of G , i.e., $z_0 = \text{RO}(\text{rt})$, where $\text{rt} = \text{MerkleTree}^{\text{RO}}(G)$. The evaluation $C_0 = \tilde{G}(z_0)$ is then included part of the commitment for $\tilde{f}$.

5.5 Summarizing Costs of Titan PCS

We summarize the costs of the Titan PCS described above. For parameters $p, q, \rho_1, \rho_2, \ell_1, \ell_2, m_1, m_2, t_1$ and t_2 (see Figure 1), such that $n = p \cdot q, \ell_1 \cdot m_1 = p, \ell_2 \cdot m_2 = q$, the costs are as follows:

- **Commit:** The cost of committing to $\tilde{f}$ is dominated by p MSMs of size q each to compute $\tilde{G}$. Other costs include encoding $\tilde{G}$ as a codeword and hashing it as a Merkle tree, which are $\tilde{O}(p) = o(n)$.
- **Eval Prover:** Discarding the polynomial evaluation, the prover incurs MSM of size p to compute partial evaluation C , group sumcheck to prove outer evaluation $\tilde{G}(\mathbf{x}) = C$ costs $\sqrt{p}$ MSMs of size $\sqrt{p}$. Finally the CSP costs k_2 MSMs of size q . Thus the total prover cost is $\approx \text{MSM}(p) + \sqrt{p} \text{MSM}(\sqrt{p}) + k_2 \cdot \text{MSM}(q) + O(p + q) \mathbb{F}$.
- **Eval Verifier:** The verifier incurs cost $\text{MSM}(t_1 \ell_1 + m_1) + t_1(m_1 + \ell_1) \mathbb{F}$ during consistency checks in Π_{outer} , and $\text{MSM}(t_2 \ell_2 + m_2) + t_2(m_2 + \ell_2) \mathbb{F}$ in Π_{inner} .
- **Eval Proof Size:** We calculate proof size as: From Lemma 5.1, communication in Π_{outer} is $t_1 \ell_1 + m_1 + 3k_1 \mathbb{G}$. The communication during Π_{inner} is (i) $k_2(2 \mathbb{G} + 2 \mathbb{F})$ during CSP rounds and (ii) $m_2 \mathbb{G} + m_2 \mathbb{F}$ to communicate reduced instance. We do not count the openings of oracle $\llbracket g \rrbracket$ towards proof-size, as we assume that verifier stores it locally. Summarising,

$$|\pi| = (t_1 \ell_1 + m_1 + m_2 + 3k_1 + 2k_2) \mathbb{G} + (m_2 + 2k_2) \mathbb{F}$$

We describe parameter settings to obtain soundness of $2^{-\lambda}$. In the list-decoding regime, choosing $\delta \approx 1 - \sqrt{\rho_1}$, gives the soundness of $\epsilon_{\text{RPE}} \approx p \cdot \rho_1^{t_1/2} + 3^{k_2} \cdot \rho_2^{t_2}$. Thus, we set $t_1 \approx 2(\lambda + \log p)/\log(1/\rho_1)$ and $t_2 \approx (\lambda + 1.6k_2)/\log(1/\rho_2)$. Thus, $t_1, t_2 = O(\lambda)$. We also set $p, q = O(\sqrt{n}), \ell_1 = \sqrt{p}/t_1, m_1 = \sqrt{p} \cdot t_1, \ell_2 = \sqrt{q}/t_2$ and $m_2 = \sqrt{q} \cdot t_2$. With these choices, we obtain verifier complexity of $O(\lambda^{3/2} \cdot n^{1/4})$ field operations and $O(\lambda n^{1/4})$ MSMs. The proof size is $O(\lambda n^{1/4})$ group elements. In Section 7, we explicitly specify parameters for each choice of m .

6 Optimizations for IOPP over Groups

In this section we describe optimizations for efficiently realizing IOPP over groups, as highlighted in Section 2. Throughout, let n denote the size of multilinear group polynomial $\tilde{g} \in \mathbb{G}^{\leq 1}[X_1, \dots, X_{\log n}]$. When used within Titan, n will be $O(\sqrt{N})$, where N is the size of the field polynomial.

6.1 Encoding Group Oracle

Given a polynomial $\tilde{g} \in \mathbb{G}^{\leq 1}[X_1, \dots, X_m]$ in evaluation basis, with evaluations $g_1, \dots, g_n$ over the hypercube, encoding g as an oracle requires evaluations $\tilde{g}(x)$ for $x \in L$. We avoid realizing $\tilde{g}$ explicitly, and instead rely on the following recursion:

$$\tilde{g}(x, x^2, \dots, x^{2^{m-1}}) = (1-x) \cdot \tilde{g}(0, x^2, \dots, x^{2^{m-1}}) + x \cdot \tilde{g}(1, x^2, \dots, x^{2^{m-1}})$$

Thus, evaluations of $\tilde{g}$ over L can be constructed in a “butterfly” like fashion from evaluations of $\tilde{g}(0, \cdot)$ and $\tilde{g}(1, \cdot)$ over domain $L^{(2)}$. The above recursion allows us to directly use evaluations $g_1, \dots, g_n$ at the final layer of the procedure. While we incur $\frac{n}{2} \log n$ scalar multiplications, we save on $\approx n \log n$ scalar multiplication required to compute $\tilde{g}(X)$ explicitly.

6.2 Efficient Group Sumcheck

Naive sumcheck over groups incurs $O(n)$ group-exponentiations for the prover. Here, we describe a variant using costs roughly equivalent of MSM of size $O(n)$, which in practice is substantially faster (optimized Pippenger’s based MSM can be 20× to 50× faster). Let $m = \log n$ and $\tilde{f}$ be an m -variate polynomial which is MLE of the function $f : \{0, 1\}^m \rightarrow \mathbb{G}$, we consider sumcheck instance of the form:

$$\sum_{x \in \{0,1\}^m} \tilde{eq}(\alpha, x) \cdot \tilde{f}(x) = \sigma \quad (11)$$

The sumcheck protocol requires the prover to send polynomials $g_i(X)$ in round i for $i \in [m]$, where the polynomial $g_i(X)$ is defined by:

$$g_i(X) := \sum_{x \in \{0,1\}^{m-i}} \tilde{eq}(\alpha, \rho_i, X, x) \cdot \tilde{f}(\rho_i, X, x) \quad (12)$$

where ρ_i denotes the vector of challenges $(r_1, \dots, r_{i-1})$ sent by the verifier in previous $i-1$ rounds. Note that ρ_1 is an empty vector. To interpolate polynomial g_i , it suffices to compute $g_i(u)$ for three distinct values of u . We discuss computation of $g_i(u)$ for $u \in \mathbb{F}$ next. We have:

$$g_i(u) := \sum_{x \in \{0,1\}^{m-i}} \tilde{eq}(\alpha, \rho_i, u, x) \cdot \tilde{f}(\rho_i, u, x) \quad (13)$$

Folklore methods incur $O(2^m) = O(n)$ cost to compute $g_i(u)$ for all $i \in [m]$. We start by defining $\tilde{h}(x) = \tilde{eq}(\alpha, x) \cdot \tilde{f}(x)$. Let $\ell = m/2$. As a first step, the prover computes the partial sums:

$$S_i(b) = \sum_{x \in \{0,1\}^{m-i}} \tilde{h}(b, x) \quad (14)$$

for all $1 \leq i \leq \ell$ where S_i is computed for all $b \in \{0, 1\}^i$. Note that computing S_i for given $b \in \{0, 1\}^i$ incurs MSM of size $O(2^{m-i})$. We compute the above sums as follows:

- (1) First compute $S_\ell(b)$ for all $b \in \{0, 1\}^\ell$. This incurs total 2^ℓ MSMs of size 2^ℓ each.

- (2) Next, we notice that $S_{i-1}(b) = S_i(b, 0) + S_i(b, 1)$ and thus the table S_{i-1} can be computed from S_i in $2^i \mathbb{G}$ additions.
- (3) Proceeding, the tables S_i for all $1 \leq i \leq \ell$ can be computed in 2^ℓ MSMs of size 2^ℓ and $O(2^\ell) = O(\sqrt{n})$ additions in $\mathbb{G}$.
- (4) As we show in Appendix B, round messages $g_i(u)$, $u \in \{0, 1, 2\}$ for the i^{th} round for $i < \ell$ can be computed using MSM of size $O(2^i)$ over table S_i as:

$$g_i(u) := \sum_{b \in \{0,1\}^i} \frac{\tilde{eq}(z, b) \cdot \tilde{eq}(z, \alpha_i)}{\tilde{eq}(\alpha_i, b)} \cdot S_i(b)$$

In the above α_i denotes the prefix $(\alpha_1, \dots, \alpha_i)$ of α .

- (5) For rounds $i \geq \ell$, the size of the polynomial reduces to $O(\sqrt{n})$, and hence the prover computes round messages in usual manner.

As it can be seen, the above scheme reduces group-exponentiation to only $O(\sqrt{n})$, while accomplishing remaining work over groups using MSMs.

6.3 Speeding up CSP prover

We reduce costs of CSP prover by using MSMs to speed up costly initial iterations. In more detail, CSP prover starts with initial commitment key $g_0 = g$ and witness $a_0 = a$ in $\mathbb{G}^n$ and $\mathbb{F}^n$ respectively. In our variant of the CSP, the commitment key g_i and the witness a_i in the i^{th} round is given by evaluations of polynomials $\tilde{g}(c_1, \dots, c_{i-1}, x)$ and $\tilde{a}(1 - c_1, \dots, 1 - c_{i-1}, x)$ where $\tilde{g}$ and $\tilde{a}$ are polynomials interpolating vectors g and a and $c_1, \dots, c_{i-1}$ are challenges so far. Let $c_i = (c_1, \dots, c_{i-1})$ and $c'_i = (1 - c_1, \dots, 1 - c_{i-1})$. In terms of these polynomials, round messages L_i and R_i in terms of the polynomials, cross term L_i can be computed via 2^{i-1} MSMs, each of size 2^{m-i} (See Appendix B). For smaller values of i , the above computation is faster than folding the key g_i using 2^i group-exponentiations. In our implementations, we use MSMs till we terminate the CSP at approximately $m/2$ rounds.

7 Implementation

We implement Titan PCS and TitanSnark titan-implementation-BB26 [2] in Rust using Pallas curve from Pasta pair of curves. The curve features a 256-bit scalar field.

Experimental Setup. All experiments were conducted on a machine running Ubuntu 24.04.3 LTS with Linux kernel 6.8.0. The system is equipped with a single AMD EPYC 9754 processor with a maximum clock frequency of 2.25 GHz, and 251 GB of RAM. To ensure fair comparison, all benchmarks were executed in a single-threaded manner by setting `RAYON_NUM_THREADS=1`, disabling parallel test execution, and pinning the process to a fixed CPU core (core 0) using `taskset`.

Comparison and Metrics. We compare the Titan commitment scheme against several transparent polynomial commitment schemes: Brake-down [31], Dory [38], Hyrax [47], Kopis [46], and WHIR [5]. We report the following metrics: (i) commitment time; (ii) evaluation time; (iii) verification time; and (iv) proof size. We note that Brake-down and WHIR are plausibly post-quantum safe, while remaining schemes, including ours rely on classical assumptions. We include them to illustrate our savings with respect to hash based schemes – at the cost of sacrificing post-quantum security.

Dory and Kopis are instantiated using the quarks-zk rust crate [42]. For Brakedown and Hyrax we use the implementations available via Arkworks poly-commit library [3], with custom benchmark modules added to measure commitment, evaluation, and verification times. WHIR is evaluated using the available implementation [48], which we instrument to separately report commitment and evaluation time. Titan is instantiated over the Pasta curve using the Pallas scalar field (256-bit) with group operations implemented via the Pallas backend. Brakedown, Dory, Hyrax, and Kopis are all instantiated over the scalar field of BLS12-381 curve, while WHIR is evaluated over a 256-bit prime field supported by its implementation.

Titan Parameters: For Titan PCS, we use $\rho_1 = \rho_2 = 1/16$ as the rates for Reed-Solomon codes for encoding the outer group polynomial and the Pedersen generators respectively. The number of outer and inner queries are set to $t_1 = 70$ and $t_2 = 35$ to achieve ≈ 128 bits of soundness. We implement Merkle path compression techniques to reduce communication overhead. The parameters for different polynomial sizes are reported in the table 1. We skew the matrix towards q (the size of generators), as querying oracle $\llbracket q \rrbracket$ has better soundness (we know it encodes a polynomial). For encoding the outer group polynomial, we roughly skew the size of folded polynomial and the size of cosets as $\sqrt{t_1 p}$ and $\sqrt{p/t_1}$ which leads to optimal proof size and verifier computation.

Table 1: Parameter settings for TITAN for different polynomial sizes. p and q denote sizes of outer and inner group polynomials. m_1 and m_2 denote size of folded outer and inner polynomials. All sizes are reported in log scale.

m	p	q	ℓ_1	m_1	ℓ_2	m_2
18	8	10	1	7	5	5
20	8	12	1	7	6	6
22	9	13	2	7	6	7
24	10	14	2	8	7	7
26	11	15	3	8	8	7

7.1 Evaluation Summary

In Table 2 we present the commit time, evaluation time, verification time and proof size of Brakedown, Dory, Hyrax, Kopis, Titan, and WHIR grouped by the number of variables $m \in \{18, 20, 22, 24, 26\}$. The lowest times for commit, eval, verify and the lowest proof size for each m are represented in teal. Section F provides a visual representation of these metrics. From the evaluation, we can see that Titan PCS meets the desiderata we started with: much smaller proof sizes than hash based schemes, while superior prover and verification performance compared to group based schemes, without sacrificing proof size significantly. While commitment and evaluation in Titan are considerably faster than Dory, the superior asymptotics of Dory verifier start catching up for larger polynomials. We provide additional break up of Titan proof size into three components: outer evaluation (IOPP), inner evaluation (CSP), and Merkle paths. Table 3 reports their sizes (in KiB) as a function of the number of variables m .

Table 2: Comparison of polynomial commitment schemes grouped by number of variables m .

m	Scheme	Commit (s)	Eval (s)	Verify (s)	Size (KiB)
18	Brakedown	0.26	0.59	0.667	5921.9
	Dory	8.75	3.87	0.186	14.9
	Hyrax	5.16	0.04	0.044	16.2
	Kopis	16.58	1.802	0.549	26.6
	Titan	6.31	0.23	0.043	24.4
	WHIR	0.23	0.419	0.002	241.8
20	Brakedown	1.13	1.12	1.128	9325.9
	Dory	29.45	8.32	0.204	16.4
	Hyrax	15.74	0.10	0.078	32.2
	Kopis	64.42	3.79	1.089	43.7
	Titan	14.02	0.69	0.053	26.3
	WHIR	1.10	1.90	0.003	271.8
22	Brakedown	5.53	2.43	2.078	15961.4
	Dory	105.25	19.18	0.222	18.0
	Hyrax	55.17	0.31	0.150	64.2
	Kopis	220.92	9.21	2.182	76.8
	Titan	44.38	1.42	0.059	34.5
	WHIR	4.70	8.00	0.003	293.8
24	Brakedown	29.73	5.65	4.112	29059.8
	Dory	376.25	50.73	0.240	19.5
	Hyrax	203.93	0.99	0.290	128.2
	Kopis	860.33	26.00	3.957	141.9
	Titan	153.92	3.74	0.080	40.9
	WHIR	21.60	33.90	0.003	322.2
26	Brakedown	137.01	15.71	8.198	55084.3
	Dory	1374.66	156.46	0.260	21.0
	Hyrax	691.56	3.47	0.549	256.2
	Kopis	2449.69	78.00	6.500	271.1
	Titan	552.11	10.87	0.118	49.8
	WHIR	93.50	144.38	0.004	346.8

Table 3: Break-up of Titan Proof Size

m	Outer Eval (KiB)	Inner Eval (KiB)	Merkle Paths (KiB)
18	8.43	2.63	13.33
20	8.43	4.75	13.06
22	12.5	8.75	13.24
24	17.06	9.0	15.16
26	24.25	9.25	15.31

7.2 TitanSnark

We also compile a variant of Spartan (specifically Microspartan) PIOP with Titan PCS to obtain a transparent SNARK which we call TitanSnark. Our SNARK TitanSnark quadratically improves proof size and verification complexity of the original scheme from [45] (the transparent variant $\text{Spartan}_{\text{DL}}$) from $O(\sqrt{n})$ to $O(\sqrt[3]{n})$. Detailed construction and evaluation appears in Sections D and E in Appendix. Concretely, the proof sizes we obtain are about $3\times$ smaller at $m = 23$.

References

- [1] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramanian. 2017. Liger: Lightweight Sublinear Arguments Without a Trusted Setup. In *ACM CCS 2017*, Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu (Eds.). ACM Press, 2087–2104. doi:10.1145/3133956.3134104
- [2] Anonymous. [n. d.]. Titan Implementation, Benchmarks. <https://anonymous.4open.science/titan-implementation-BB26>.
- [3] Arkworks-rs. [n. d.]. poly-commit Library. <https://github.com/arkworks-rs/poly-commit>.
- [4] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. 2024. STIR: Reed-Solomon Proximity Testing with Fewer Queries. In *CRYPTO 2024, Part X (LNCS, Vol. 14929)*, Leonid Reyzin and Douglas Stebila (Eds.). Springer, Cham, 380–413. doi:10.1007/978-3-031-68403-6_12
- [5] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. 2025. WHIR: Reed-Solomon Proximity Testing with Super-Fast Verification. In *EUROCRYPT 2025, Part IV (LNCS, Vol. 15604)*, Serge Fehr and Pierre-Alain Fouque (Eds.). Springer, Cham, 214–243. doi:10.1007/978-3-031-91134-7_8
- [6] Sanjeev Arora, Carsten Lund, Rajeev Motwani, Madhu Sudan, and Mario Szegedy. 1992. Proof Verification and Hardness of Approximation Problems. In *33rd FOCS*. IEEE Computer Society Press, 14–23. doi:10.1109/SFCS.1992.267823
- [7] Sanjeev Arora and Shmuel Safra. 1992. Probabilistic Checking of Proofs; A New Characterization of NP. In *33rd FOCS*. IEEE Computer Society Press, 2–13. doi:10.1109/SFCS.1992.267824
- [8] Arasu Arun, Chaya Ganesh, Satya V. Lokam, Tushar Mopuri, and Sriram Sridhar. 2023. Dew: A Transparent Constant-Sized Polynomial Commitment Scheme. In *PKC 2023, Part II (LNCS, Vol. 13941)*, Alexandra Boldyreva and Vladimir Kolesnikov (Eds.). Springer, Cham, 542–571. doi:10.1007/978-3-031-31371-4_19
- [9] Thomas Attema and Ronald Cramer. 2020. Compressed Σ -Protocol Theory and Practical Application to Plug & Play Secure Algorithmics. In *CRYPTO 2020, Part III (LNCS, Vol. 12172)*, Daniele Micciancio and Thomas Ristenpart (Eds.). Springer, Cham, 513–543. doi:10.1007/978-3-030-56877-1_18
- [10] László Babai, Lance Fortnow, Leonid A. Levin, and Mario Szegedy. 1991. Checking Computations in Polylogarithmic Time. In *23rd ACM STOC*. ACM Press, 21–31. doi:10.1145/103418.103428
- [11] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. 2018. Fast Reed-Solomon Interactive Oracle Proofs of Proximity. In *ICALP 2018 (LIPIcs, Vol. 107)*, Ioannis Chatzigiannakis, Christos Kallamanis, Dániel Marx, and Donald Sannella (Eds.). Schloss Dagstuhl, 14:1–14:17. doi:10.4230/LIPIcs.ICALP.2018.14
- [12] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. 2018. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Report 2018/046. <https://eprint.iacr.org/2018/046>
- [13] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. 2019. Aurora: Transparent Succinct Arguments for R1CS. In *EUROCRYPT 2019, Part I (LNCS, Vol. 11476)*, Yuval Ishai and Vincent Rijmen (Eds.). Springer, Cham, 103–128. doi:10.1007/978-3-030-17653-2_4
- [14] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. 2016. Interactive Oracle Proofs. In *TCC 2016-B, Part II (LNCS, Vol. 9986)*, Martin Hirt and Adam D. Smith (Eds.). Springer, Berlin, Heidelberg, 31–60. doi:10.1007/978-3-662-53644-5_2
- [15] Eli Ben-Sasson, Lior Goldberg, Swastik Kopparty, and Shubhangi Saraf. 2020. DEEP-FRI: Sampling Outside the Box Improves Soundness. In *ITCS 2020*, Thomas Vidick (Ed.), Vol. 151. LIPIcs, 5:1–5:32. doi:10.4230/LIPIcs.ITCS.2020.5
- [16] Jonathan Bootle, Andrea Cerulli, Pyrrhos Chaidos, Jens Groth, and Christophe Petit. 2016. Efficient Zero-Knowledge Arguments for Arithmetic Circuits in the Discrete Log Setting. In *EUROCRYPT 2016, Part II (LNCS, Vol. 9666)*, Marc Fischlin and Jean-Sébastien Coron (Eds.). Springer, Berlin, Heidelberg, 327–357. doi:10.1007/978-3-662-49896-5_12
- [17] Jonathan Bootle, Alessandro Chiesa, Yuncong Hu, and Michele Orrù. 2022. Gemini: Elastic SNARKs for Diverse Environments. In *EUROCRYPT 2022, Part II (LNCS, Vol. 13276)*, Orr Dunkelman and Stefan Dziembowski (Eds.). Springer, Cham, 427–457. doi:10.1007/978-3-031-07085-3_15
- [18] Jonathan Bootle, Alessandro Chiesa, and Katerina Sotiraki. 2021. Sumcheck Arguments and Their Applications. In *CRYPTO 2021, Part I (LNCS, Vol. 12825)*, Tal Malkin and Chris Peikert (Eds.). Springer, Cham, Virtual Event, 742–773. doi:10.1007/978-3-030-84242-0_26
- [19] Benedikt Bünz, Mary Maller, Pratyush Mishra, Nirvan Tyagi, and Psi Vesely. 2021. Proofs for Inner Pairing Products and Applications. In *ASIACRYPT 2021, Part III (LNCS, Vol. 13092)*, Mehdi Tibouchi and Huaxiong Wang (Eds.). Springer, Cham, 65–97. doi:10.1007/978-3-030-92078-4_3
- [20] Benedikt Bünz, Tushar Mopuri, Alireza Shirzad, and Sriram Sridhar. 2025. DewTwo: A Transparent PCS with Quasi-Linear Prover, Logarithmic Verifier and 4.5KB Proofs from Falsifiable Assumptions. In *CRYPTO 2025, Part VI (LNCS, Vol. 16005)*, Yael Tauman Kalai and Seny F. Kamara (Eds.). Springer, Cham, 549–583. doi:10.1007/978-3-032-01887-8_18
- [21] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. 2023. HyperPlonk: Plonk with Linear-Time Prover and High-Degree Custom Gates. In *EUROCRYPT 2023, Part II (LNCS, Vol. 14005)*, Carmit Hazay and Martijn Stam (Eds.). Springer, Cham, 499–530. doi:10.1007/978-3-031-30617-4_17
- [22] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Psi Vesely, and Nicholas P. Ward. 2020. Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS. In *EUROCRYPT 2020, Part I (LNCS, Vol. 12105)*, Anne Canteaut and Yuval Ishai (Eds.). Springer, Cham, 738–768. doi:10.1007/978-3-030-45721-1_26
- [23] Elizabeth Crites and Alistair Stewart. 2025. On Reed–Solomon Proximity Gaps Conjectures. Cryptology ePrint Archive, Paper 2025/2046. <https://eprint.iacr.org/2025/2046>
- [24] Liam Eagen and Ariel Gabizon. 2025. MERCURY: A multilinear Polynomial Commitment Scheme with constant proof size and no prover FFTs. Cryptology ePrint Archive, Report 2025/385. <https://eprint.iacr.org/2025/385>
- [25] Liam Eagen and Ariel Gabizon. 2025. Revisiting the IPA-sumcheck connection. Cryptology ePrint Archive, Paper 2025/1325. <https://eprint.iacr.org/2025/1325>
- [26] Uriel Feige, Shafi Goldwasser, László Lovász, Shmuel Safra, and Mario Szegedy. 1996. Interactive proofs and the hardness of approximating cliques. *Journal of the ACM (JACM)* 43, 2 (1996), 268–292.
- [27] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. 2019. PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge. Cryptology ePrint Archive, Report 2019/953. <https://eprint.iacr.org/2019/953>
- [28] Nicolas Gailly, Mary Maller, and Anca Nitulescu. 2022. SnarkPack: Practical SNARK Aggregation. In *FC 2022 (LNCS, Vol. 13411)*, Ittay Eyal and Juan A. Garay (Eds.). Springer, Cham, 203–229. doi:10.1007/978-3-031-18283-9_10
- [29] Chaya Ganesh, Sikhar Patranabis, Shubh Prakash, and Nitin Singh. 2024. GAPP: Generic Aggregation of Polynomial Protocols. Cryptology ePrint Archive, Report 2024/1685. <https://eprint.iacr.org/2024/1685>
- [30] Chaya Ganesh, Sikhar Patranabis, and Nitin Singh. 2025. Samaritan: Linear-time Prover SNARK from New Multilinear Polynomial Commitments. Cryptology ePrint Archive, Report 2025/419. <https://eprint.iacr.org/2025/419>
- [31] Alexander Golovnev, Jonathan Lee, Srinath T. V. Setty, Justin Thaler, and Riad S. Wahby. 2023. Brakedown: Linear-Time and Field-Agnostic SNARKs for R1CS. In *CRYPTO 2023, Part II (LNCS, Vol. 14082)*, Helena Handschuh and Anna Lysyanskaya (Eds.). Springer, Cham, 193–226. doi:10.1007/978-3-031-38545-2_7
- [32] Yanpei Guo, Xuanming Liu, Kexi Huang, Wenjie Qu, Tianyang Tao, and Jiaheng Zhang. 2024. DeepFold: Efficient Multilinear Polynomial Commitment from Reed-Solomon Code and Its Application to Zero-knowledge Proofs. Cryptology ePrint Archive, Report 2024/1595. <https://eprint.iacr.org/2024/1595>
- [33] Ulrich Haböck. 2022. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Report 2022/1530. <https://eprint.iacr.org/2022/1530>
- [34] Ulrich Haböck. 2025. A note on mutual correlated agreement for Reed-Solomon codes. Cryptology ePrint Archive, Paper 2025/2110. <https://eprint.iacr.org/2025/2110>
- [35] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. 2010. Constant-Size Commitments to Polynomials and Their Applications. In *ASIACRYPT 2010 (LNCS, Vol. 6477)*, Masayuki Abe (Ed.). Springer, Berlin, Heidelberg, 177–194. doi:10.1007/978-3-642-17373-8_11
- [36] Assimakis A. Kattis, Konstantin Panarin, and Alexander Vlasov. 2022. RedShift: Transparent SNARKs from List Polynomial Commitments. In *ACM CCS 2022*, Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi (Eds.). ACM Press, 1725–1737. doi:10.1145/3548606.3560657
- [37] Tohru Kohrita and Patrick Towa. 2023. Zeromorph: Zero-Knowledge Multilinear-Evaluation Proofs from Homomorphic Univariate Commitments. Cryptology ePrint Archive, Report 2023/917. <https://eprint.iacr.org/2023/917>
- [38] Jonathan Lee. 2021. Dory: Efficient, Transparent Arguments for Generalised Inner Products and Polynomial Commitments. In *TCC 2021, Part II (LNCS, Vol. 13043)*, Kobbi Nissim and Brent Waters (Eds.). Springer, Cham, 1–34. doi:10.1007/978-3-030-90453-1_1
- [39] Mary Maller, Sean Bowe, Markulf Kohlweiss, and Sarah Meiklejohn. 2019. Sonic: Zero-Knowledge SNARKs from Linear-Size Universal and Updatable Structured Reference Strings. In *ACM CCS 2019*, Lorenzo Cavallaro, Johannes Kinder, Xiaofeng Wang, and Jonathan Katz (Eds.). ACM Press, 2111–2128. doi:10.1145/3319535.3339817
- [40] Microsoft. [n. d.]. Spartan: High Speed zkSNARKs Without Trusted Setup. <https://github.com/microsoft/Spartan>.
- [41] Charalampos Papamanthou, Elaine Shi, and Roberto Tamassia. 2013. Signatures of Correct Computation. In *TCC 2013 (LNCS, Vol. 7785)*, Amit Sahai (Ed.). Springer, Berlin, Heidelberg, 222–242. doi:10.1007/978-3-642-36594-2_13
- [42] Quarks ZK Developers. [n. d.]. quarks-zk Rust Crate. <https://crates.io/crates/quarks-zk/0.1.5>.
- [43] Omer Reingold, Guy N. Rothblum, and Ron D. Rothblum. 2016. Constant-round interactive proofs for delegating computation. In *48th ACM STOC*, Daniel Wichs and Yishay Mansour (Eds.). ACM Press, 49–62. doi:10.1145/2897518.2897652
- [44] István András Seres and Péter Burcsi. 2025. Behemoth: Transparent Polynomial Commitment Scheme with Constant Opening Proof Size and Verifier Time. In *AFRICRYPT 25 (LNCS, Vol. 15651)*, Abderrahmane Nitaj, Svetla Petkova-Nikova, and Vincent Rijmen (Eds.). Springer, Cham, 343–369. doi:10.1007/978-3-031-97260-7_16
- [45] Srinath Setty. 2020. Spartan: Efficient and General-Purpose zkSNARKs Without Trusted Setup. In *CRYPTO 2020, Part III (LNCS, Vol. 12172)*, Daniele Micciancio

- and Thomas Ristenpart (Eds.). Springer, Cham, 704–737. doi:10.1007/978-3-030-56877-1_25
- [46] Srinath Setty and Jonathan Lee. 2020. Quarks: Quadruple-efficient transparent zkSNARKs. Cryptology ePrint Archive, Report 2020/1275. <https://eprint.iacr.org/2020/1275>
- [47] Riad S. Wahby, Ioanna Tzialla, abhi shelat, Justin Thaler, and Michael Wal-fish. 2018. Doubly-Efficient zkSNARKs Without Trusted Setup. In *2018 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, 926–943. doi:10.1109/SP.2018.00060
- [48] WizardOfMenlo. [n. d.]. WHIR Reference Implementation. <https://github.com/WizardOfMenlo/whir>.
- [49] Hadas Zeilberger, Binyi Chen, and Ben Fisch. 2024. BaseFold: Efficient Field-Agnostic Polynomial Commitment Schemes from Foldable Codes. In *CRYPTO 2024, Part X (LNCS, Vol. 14929)*, Leonid Reyzin and Douglas Stebila (Eds.). Springer, Cham, 138–169. doi:10.1007/978-3-031-68403-6_5
- [50] Yuncong Zhang. 2025. Revisiting Sum-check-based Polynomial Commitment Schemes. Cryptology ePrint Archive, Paper 2025/2249. <https://eprint.iacr.org/2025/2249>
- [51] Jiaxing Zhao, Srinath Setty, and Weidong Cui. 2024. MicroNova: Folding-based arguments with efficient (on-chain) verification. Cryptology ePrint Archive, Report 2024/2099. <https://eprint.iacr.org/2024/2099>

Ethical Considerations

The implementation and benchmarks reported in this paper were carried out on synthetic inputs generated for performance analysis. Our work does not involve any human subjects, user or personal data, real world deployed systems.

Acknowledgement of Generative AI Usage

Generative AI tools (specifically, Claude) were used for literature search assistance, coding assistance and debugging. All AI generated outputs were manually verified by the authors. All technical contributions, proofs, and performance evaluations are the authors' own work.

A Open Science Policy

We provide all artifacts related to Titan commitment scheme, such as its implementation, benchmarking, plotting scripts, precomputed results (CSV and PNG files), and documentation for reproducing experiments via an anonymized repository ([URL: anonymous.4open.science/r/titan-implementation-BB26](https://anonymous.4open.science/r/titan-implementation-BB26)) [2]. Experiments are implemented in Rust and executed in release mode under controlled conditions: single-core execution and disabled parallelism ensuring consistent benchmarking. Non-Linux platforms are supported without CPU pinning with potentially higher numbers. We compare against Brakedown, Dory, Hyrax, Kopis, and Whir. Implementations are included wherever possible, and otherwise scripts and instructions are provided to reproduce results from external repositories, all under identical experimental settings. Within the repository, the Titan and Kopis implementations are located in `src/protocols`, the Dory implementation is in `src/`, the Whir benchmarking script and all benchmarking outputs (CSV files and generated plots) are available in `src/benchmarking`.

To reproduce results for Titan, one can run the provided scaling benchmark:

```
taskset -c 0 env RAYON_NUM_THREADS=1 \
RUST_TEST_THREADS=1 \
cargo test titan_scaling_experiment \
--release -- --ignored --nocapture \
--test-threads=1 \
| grep '[0-9]' > titan_clean.csv
```

which produces CSV outputs containing input size, commit time, evaluation time, verification time, and proof size. Plots can then be regenerated using the included Python scripts. Baseline schemes (Brakedown, Dory, Hyrax, Kopis, and Whir) follow the same command structure and configuration. Detailed instructions for all experiments are included in the repository. To execute the TitanSnark protocol, the implementation is provided in `src/protocols/spartan_titan.rs`. A complete end-to-end test (covering setup, indexing, proof generation, and verification over a synthetic R1CS instance) is included and can be executed via:

```
cargo test \
test_spartan_titan_index \
--release -- --nocapture
```

This test generates a random satisfiable R1CS instance, runs the setup, index construction, proof generation, and verification procedures, and reports timing and proof size metrics. The parameters (e.g., constraint size and number of queries) can be adjusted directly in the test code.

B Deferred Proofs

THEOREM B.1 (RESTATED). *The scheme Titan PCS consisting of algorithms (Setup, Commit, Open, Eval) as in Figure 5 for polynomials in $\mathbb{F}^{\leq 1}[X_1, \dots, X_m]$ satisfies completeness, binding, and knowledge soundness with random point evaluation with soundness error $\epsilon_{\text{RPE}} \leq (p+2)(1-\delta)^{t_1} + 3^{k_2} \rho_2^{t_2} + \epsilon_{\text{CSP}} + \epsilon_{\text{DLOG}}$. Here t_1 and t_2 denote number of consistency queries in Π_{outer} and Π_{inner} respectively, while ϵ_{CSP} and ϵ_{DLOG} denote soundness of standard CSP and adversary's success probability in breaking discrete-log in $\mathbb{G}$ respectively.*

PROOF. We skip completeness as it follows trivially by construction. To prove binding, suppose PPT adversary $\mathcal{A}$ outputs $(\llbracket G \rrbracket, \tilde{f}_0, \tilde{f}_1)$ in the binding game. If $\tilde{f}_0$ and $\tilde{f}_1$ are two distinct polynomials in $\mathbb{F}^{\leq 1}[X_1, \dots, X_m]$ satisfying **Open**(pp, $G; \tilde{f}_i$) = 1 for $i = 0, 1$; by definition of **Open** we must have $\Delta(G, \tilde{G}_i) < \delta$ for $\tilde{G}_i = \text{Com}(\tilde{f}_i; \mathbf{g})$. Since $\delta < (1-\rho_1)/2$, by triangle inequality we must have $\Delta(\tilde{G}_0, \tilde{G}_1) < 1 - \rho_1$. As $\tilde{G}_0$ and $\tilde{G}_1$ are codewords (we identify polynomials with their respective encodings) in $\text{RS}[\mathbb{G}, L_1, \ell_p]$ which has minimum distance $1 - \rho_1$, we must have $\tilde{G}_0 = \tilde{G}_1$. Thus $(\tilde{f}_0, \tilde{f}_1)$ break the “binding” of Pedersen commitment scheme, and hence any PPT adversary $\mathcal{A}$ outputs them with negligible probability.

We now prove knowledge-soundness with random point evaluation. Informally, the outer IOPP Π_{outer} binds the prover to a unique multilinear polynomial $\tilde{G}$ with coefficients $G_1, \dots, G_p \in \mathbb{G}$. Subsequently, for each evaluation claim for $\mathbf{z} = (\mathbf{x}, \mathbf{y})$, the prover shows knowledge of opening for a linear combination of G_i 's determined by $\mathbf{x}$, specifically for $C = \sum_{i=1}^m \tilde{e}q(\mathbf{x}, \langle i \rangle) \cdot G_i$. These openings for several random $\mathbf{x}$ can be combined to obtain openings for each G_i , $i \in [t]$, which determine the “rows” of the polynomial $\tilde{f}$. The formal proof follows.

Let $\mathcal{A}$ denote the adversary in the knowledge soundness game in Definition 3.3. Let

$$(G, \mathbf{z}, \sigma, \text{tr} = (C, \dots))$$

be a successful transcript produced by $\mathcal{A}^5$. In the above, we use tr to denote the transcript of the evaluation protocol $\Pi_{\text{Eval}}(\llbracket G \rrbracket, \mathbf{z}, \sigma)$. The extractor Ext works as follows: It repeatedly rewinds $\mathcal{A}$ to output p accepting transcripts

$$(G, \mathbf{z}_i, \sigma_i, \text{tr}_i = (C_i, \dots)), i \in [p]$$

By the *Forking Lemma* [16, Lemma 1] for interactive protocols, Ext runs in expected polynomial time. Let $\mathbf{z}_i = (\mathbf{x}_i, \mathbf{y}_i)$ where $\mathbf{x}_i \in \mathbb{F}^{\ell_p}$, $\mathbf{y}_i \in \mathbb{F}^{\ell_q}$. Since tr_i contains an accepting transcript for $\Pi_{\text{inner}}(C_i, \mathbf{y}_i, \sigma_i)$, invoking extractor $\text{Ext}_{\text{inner}}^{\mathcal{A}}$ for the protocol Π_{inner} , we obtain $(a_{i1}, \dots, a_{iq})$ such that: $\sum_{j=1}^q a_{ij} \cdot g_j = C_i$ for $i \in [p]$. Assuming that C_i 's are evaluations of some multilinear polynomial $\tilde{G}$ at points $\mathbf{x}_i$, the extractor Ext sets up the following system of equations with unknowns G_i , $i \in [t]$:

$$\begin{bmatrix} \tilde{eq}(\mathbf{x}_1, \langle 1 \rangle) & \tilde{eq}(\mathbf{x}_1, \langle 2 \rangle) & \cdots & \tilde{eq}(\mathbf{x}_1, \langle p \rangle) \\ \tilde{eq}(\mathbf{x}_2, \langle 1 \rangle) & \tilde{eq}(\mathbf{x}_2, \langle 2 \rangle) & \cdots & \tilde{eq}(\mathbf{x}_2, \langle p \rangle) \\ \vdots & \vdots & \ddots & \vdots \\ \tilde{eq}(\mathbf{x}_p, \langle 1 \rangle) & \tilde{eq}(\mathbf{x}_p, \langle 2 \rangle) & \cdots & \tilde{eq}(\mathbf{x}_p, \langle p \rangle) \end{bmatrix} \begin{bmatrix} G_1 \\ G_2 \\ \vdots \\ G_p \end{bmatrix} = \begin{bmatrix} C_1 \\ C_2 \\ \vdots \\ C_p \end{bmatrix}$$

$$= \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1q} \\ a_{21} & a_{22} & \cdots & a_{2q} \\ \vdots & \vdots & \ddots & \vdots \\ a_{p1} & a_{p2} & \cdots & a_{pq} \end{bmatrix} \begin{bmatrix} g_1 \\ g_2 \\ \vdots \\ g_q \end{bmatrix}$$

As shown in Lemma B.2 (reproduced from [49]) the matrix consisting of evaluations $\tilde{eq}(\mathbf{x}_i, \langle j \rangle)$ for uniformly sampled $\mathbf{x}_i$ is invertible with overwhelming probability. Thus, Ext can compute $p \times q$ matrix $\mathbf{F}$ such that $\mathbf{F} \cdot \mathbf{g} = \mathbf{G}$. The extractor outputs the unique multilinear polynomial $\tilde{f}$ satisfying $\tilde{f}(\langle i \rangle, \langle j \rangle) = F_{ij}$ for $i \in [p]$, $j \in [q]$. Note that by construction G_i commits to the i^{th} row polynomial of $\tilde{f}$. We now argue that $\tilde{f}$ satisfies the relation $\mathcal{R}_{\text{ev}}$ whenever $\mathcal{A}$ succeeds in the evaluation protocol. Let GoodOrac denote the event that $\Delta(G, \tilde{G}) < \delta$ for some (unique) polynomial $\tilde{G}$. Let Fail denote the event $b = 1 \wedge \mathcal{R}_{\text{Eval}}(\text{pp}, \llbracket G \rrbracket, \mathbf{z}, \sigma; \tilde{f}) = 0$ for $\tilde{f}$ output by Ext . We bound $\Pr[\text{Fail}]$ as follows:

Case 1: We consider $\Pr[\text{Fail} \wedge \text{GoodOrac}]$. Let $\tilde{G}'$ be the unique polynomial in the δ -ball around G with and let $\tilde{G}$ be the polynomial obtained by solving the system of equations (we ignore the probability that the system of equations becomes degenerate). We decompose the event GoodOrac into two disjoint events $G_1 \equiv \text{GoodOrac} \wedge \tilde{G} = \tilde{G}'$ and $G_2 \equiv \text{GoodOrac} \wedge \tilde{G} \neq \tilde{G}'$. We first write:

$$\Pr[\text{Fail} \wedge \text{GoodOrac}] \leq \Pr[\text{Fail} \wedge G_1] + \Pr[G_2] \quad (15)$$

Since Fail is a subset of the event $\tilde{f}(\mathbf{z}) \neq \sigma$, we upper bound the first summand as follows:

$$\begin{aligned} \Pr[\text{Fail} \wedge G_1] &\leq \Pr[\tilde{f}(\mathbf{z}) \neq \sigma \wedge G_1] \\ &\leq \Pr[\tilde{f}(\mathbf{z}) \neq \sigma \wedge G_1 \wedge \tilde{G}'(\mathbf{x}) \neq C] \\ &\quad + \Pr[\tilde{f}(\mathbf{z}) \neq \sigma \wedge G_1 \wedge \tilde{G}'(\mathbf{x}) = C] \\ &\leq (1 - \delta)^{t_1} + 3^{k_2} \cdot \rho_2^{t_2} + \varepsilon + \varepsilon_{\text{DLOG}} \end{aligned}$$

In the above, we used $\Pr[\tilde{f}(\mathbf{z}) \neq \sigma \wedge G_1 \wedge \tilde{G}'(\mathbf{x}) \neq C] \leq (1 - \delta)^{t_1}$ as it corresponds to verifier randomness in Π_{outer} which leads to

⁵Note that entire oracle is considered part of transcript in an IOP. During BCS compilation, Merkle tree extractor can be used to extract consistent oracle

soundness break. Next we bound the probability of the second event where $\tilde{G}'(\mathbf{x}) = C$ and $\tilde{f}(\mathbf{z}) \neq \sigma$. Note that by construction of $\tilde{f}$ by Ext , $C = \tilde{G}'(\mathbf{x})$ is a Pedersen commitment of $\tilde{f}(\mathbf{x}, \mathbf{Y})$. Since, the transcript for $\Pi_{\text{inner}}(C, \mathbf{y}, \sigma)$ accepts, using the extractor $\text{Ext}_{\text{inner}}^{\mathcal{A}}$, the polynomial $\tilde{a}(Y)$ satisfying $\text{Com}(\tilde{a}; \mathbf{g}) = C$ and $\tilde{a}(\mathbf{y}) = C$ can be extracted except with probability $3^{k_2} \cdot \rho_2^{t_2} + \varepsilon$. Assuming, $\tilde{a}$ is extracted we must have $\tilde{a}(\mathbf{Y}) \neq \tilde{f}(\mathbf{x}, \mathbf{Y})$ as they have different evaluations at $\mathbf{y}$. Since both the polynomials have the same commitment, this constitutes breaking the discrete-log assumption in $\mathbb{G}$, whose probability we upper bound by $\varepsilon_{\text{DLOG}}$.

Now consider $\Pr[G_2]$. For G_2 to occur, it must be the case that $\tilde{G}'(\mathbf{x}_i) \neq C_i$ for some $i \in [p]$. This is so, because ignoring the probability that the system of equations is degenerate, the solution must be unique. Thus, for some i , G is δ -far from all polynomials $\tilde{G}$ satisfying $\tilde{G}(\mathbf{x}_i) = C_i$ (as $\tilde{G}'$ was the only candidate in the δ -ball, which does not satisfy the evaluation constraint). By Lemma 5.1, since the transcript tr_i accepts, we must have $\Pr[\tilde{G}'(\mathbf{x}_i) \neq C_i] \leq (1 - \delta)^{t_1}$ (as earlier, this corresponds to verifier randomness in Π_{outer} leading to soundness break). Thus $\Pr[G_2] \leq p \cdot (1 - \delta)^{t_1}$. Combining the two cases, we have $\Pr[\text{Fail} \wedge \text{GoodOrac}] \leq (p + 1) \cdot (1 - \delta)^{t_1} + 3^{k_2} \cdot \rho_2^{t_2} + \varepsilon + \varepsilon_{\text{DLOG}}$.

Case 2: Consider $\Pr[\text{Fail} \wedge \neg \text{GoodOrac}]$. In this case, since G is δ -far from the code $\text{RS}(\mathbb{G}, L_1, \ell_p)$, we see that $\Pr[\text{Fail} \wedge \neg \text{GoodOrac}] \leq \Pr[b = 1 \wedge \neg \text{GoodOrac}] \leq (1 - \delta)^{t_1}$ by the soundness of Π_{outer} .

Thus, we have shown that $\Pr[\text{Fail}] \leq (p + 2)(1 - \delta)^{t_1} + 3^{k_2} \rho_2^{t_2} + \varepsilon + \varepsilon_{\text{DLOG}}$. $\square$

LEMMA B.2. [49] For randomly chosen $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_p \leftarrow \mathbb{F}^p$ the vectors $(\tilde{eq}(\langle j \rangle, \mathbf{x}_1)_{j \in [p]}, (\tilde{eq}(\langle j \rangle, \mathbf{x}_2)_{j \in [p]}, \dots, (\tilde{eq}(\langle j \rangle, \mathbf{x}_p)_{j \in [p]})$ are linearly independent with overwhelming probability.

PROOF. Let us define the event E_i , where $E_i = 1$ if i^{th} vector $(\tilde{eq}(\langle j \rangle, \mathbf{x}_i)_{j \in [p]})$ is NOT in the span of the first $i - 1$ vectors assuming all of the $i - 1$ vectors are linearly independent. i.e $E_i = 1$ denotes that all the i vectors over the random choice of $\mathbf{x}_1, \dots, \mathbf{x}_i$ are linearly independent. Then,

$$\begin{aligned} &\Pr[\text{all } p \text{ vectors are linearly independent}] \\ &= \Pr[E_1 = 1] \cdot \Pr[E_2 = 1 | E_1 = 1] \cdots \Pr[E_p = 1 | E_{p-1} = 1] \end{aligned}$$

For any $i \in [1, p]$ and any $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_p \in \mathbb{F}^p$ we will show that $\Pr[E_i = 1 | E_{i-1} = 1] \geq 1 - \frac{\log p}{|\mathbb{F}|}$ over the random choice of $\mathbf{x}_i \leftarrow \mathbb{F}$. Let $M \in \mathbb{F}^{(i-1) \times p}$ be the matrix consisting of the vectors $(\tilde{eq}(\langle j \rangle, \mathbf{x}_1)_{j \in [p]}, \dots, (\tilde{eq}(\langle j \rangle, \mathbf{x}_{i-1})_{j \in [p]})$ as rows. We get $M' \in \mathbb{F}^{i \times p}$ by adding $(\tilde{eq}(\langle j \rangle, \mathbf{x}_i)_{j \in [p]})$ as a row. Now $E_{i-1} = 1$ implies that all the $i - 1$ vectors are linearly independent and rank of M is $i - 1$. If $E_i = 1$ then rank of M' will be $i = \text{rank}(M) + 1$ which implies $\text{kernel}(M') \subsetneq \text{kernel}(M)$. So, we want to analyze the probability that there is one zero vector in $\text{kernel}(M)$ which does not belong to $\text{kernel}(M')$. From the Rank Nullity theorem we know that $|\text{kernel}(M)| = |\mathbb{F}^{p-i}| > 1$. So we can pick a nonzero vector $G = (G_1, \dots, G_p) \in \mathbb{G}^p \in \text{kernel}(M)$ and check the probability that G is also in $\text{kernel}(M')$ i.e $M' \cdot G = 0^i$ which is the same as the probability that $\langle (G_1, \dots, G_p), (\tilde{eq}(\langle j \rangle, \mathbf{x}_i)_{j \in [p]}) \rangle = G(\mathbf{x}_i) = 0$. G being a log p -variate multilinear polynomial, the probability that G is also in $\text{kernel}(M')$ is $\frac{\log p}{|\mathbb{F}|}$ by Schwartz-Zippel lemma. So, the

probability that rank of $M' = i + 1$ is at least $1 - \frac{\log p}{|\mathbb{F}|}$, which implies

$$\Pr[E_i = 1 | E_{i-1} = 1] = 1 - \frac{\log p}{|\mathbb{F}|}$$

So, the probability that all the p vectors over the random choice of the p points are linearly independent is

$$\left(1 - \frac{\log p}{|\mathbb{F}|}\right)^p \geq 1 - p \log p / |\mathbb{F}|$$

□

Computing round messages in group sumcheck. We now consider cost of generating sumcheck messages for each round for the sumcheck protocol in Section 6.2. For round $i < \ell$, we have: (Let $\mathbf{z} = (r_1, \dots, r_{i-1}, u)$ denote the vector containing u appended to challenges sent).

$$\begin{aligned} g_i(u) &= \sum_{\mathbf{x} \in \{0,1\}^{m-i}} \tilde{eq}(\alpha, \mathbf{z}, \mathbf{x}) \cdot \tilde{f}(\mathbf{z}, \mathbf{x}) \\ &= \sum_{\mathbf{x} \in \{0,1\}^{m-i}} \tilde{eq}(\alpha, \mathbf{z}, \mathbf{x}) \sum_{\mathbf{b} \in \{0,1\}^i} \tilde{eq}(\mathbf{z}, \mathbf{b}) \cdot \tilde{f}(\mathbf{b}, \mathbf{x}) \\ &= \sum_{\mathbf{b} \in \{0,1\}^i} \tilde{eq}(\mathbf{z}, \mathbf{b}) \sum_{\mathbf{x} \in \{0,1\}^{m-i}} \tilde{eq}(\alpha, \mathbf{z}, \mathbf{x}) \tilde{f}(\mathbf{b}, \mathbf{x}) \\ &= \sum_{\mathbf{b} \in \{0,1\}^i} \tilde{eq}(\mathbf{z}, \mathbf{b}) \sum_{\mathbf{x} \in \{0,1\}^{m-i}} \tilde{eq}(\alpha, \mathbf{b}, \mathbf{x}) \left(\frac{\tilde{eq}(\alpha, \mathbf{z}, \mathbf{x})}{\tilde{eq}(\alpha, \mathbf{b}, \mathbf{x})} \right) \tilde{f}(\mathbf{b}, \mathbf{x}) \\ &= \sum_{\mathbf{b} \in \{0,1\}^i} \frac{\tilde{eq}(\mathbf{z}, \mathbf{b}) \cdot \tilde{eq}(\mathbf{z}, \alpha_i)}{\tilde{eq}(\alpha_i, \mathbf{b})} \cdot S_i(\mathbf{b}) \end{aligned} \quad (16)$$

In the above, we have $\alpha_i = \alpha[1 : i]$ and we use the fact that:

$$\frac{\tilde{eq}(\alpha, \mathbf{z}, \mathbf{x})}{\tilde{eq}(\alpha, \mathbf{b}, \mathbf{x})} = \frac{\tilde{eq}(\alpha_i, \mathbf{z})}{\tilde{eq}(\alpha_i, \mathbf{b})} \quad (17)$$

Given the precomputed values of $S_i(\mathbf{b})$, the above can be computed using MSM of size $O(2^i)$. Thus, the total cost to compute first ℓ round messages is $O(2^\ell) = O(\sqrt{n})$ MSM. The final ℓ messages can be computed by usual techniques, as the size of the polynomial after fixing the first ℓ challenges is $O(\sqrt{n})$ and so it incurs $O(\sqrt{n})$ group exponentiations. The total complexity of a sum-check for group polynomial is therefore $\sqrt{n}$ MSM($\sqrt{n}$) + $O(\sqrt{n})$ group-exp.

Computing Round Messages in CSP. We show how round message L_i (a cross-commitment) in a CSP can be expressed in terms of group polynomials. This interpretation allows it to be computed via 2^{i-1} MSMs, each of size 2^{m-i} .

$$\begin{aligned} L_i &= \sum_{\mathbf{x} \in \{0,1\}^{m-i}} \tilde{a}(\mathbf{c}'_i, 1, \mathbf{x}) \cdot \tilde{g}(\mathbf{c}_i, 0, \mathbf{x}) \\ &= \sum_{\mathbf{x} \in \{0,1\}^{m-i}} \tilde{a}(\mathbf{c}'_i, 1, \mathbf{x}) \sum_{\mathbf{b} \in \{0,1\}^{i-1}} \tilde{eq}(\mathbf{c}_i, \mathbf{b}) \cdot \tilde{g}(\mathbf{b}, 0, \mathbf{x}) \\ &= \sum_{\mathbf{b} \in \{0,1\}^{i-1}} \tilde{eq}(\mathbf{c}_i, \mathbf{b}) \sum_{\mathbf{x} \in \{0,1\}^{m-i}} \tilde{a}(\mathbf{c}'_i, 1, \mathbf{x}) \cdot \tilde{g}(\mathbf{b}, 1, \mathbf{x}) \end{aligned}$$

C Extended Preliminaries

In this section, we include a self-contained description of tools and definitions from prior work to help the reader understand our construction.

C.1 WHIR Iteration

We now describe a single iteration of WHIR which reduces testing proximity of a function to $\text{CRS}[\mathbb{F}, L, m, \tilde{w}, \sigma]$ to testing proximity of a function to $\text{CRS}[\mathbb{F}, L^{(2^k)}, m-k, \tilde{w}', \sigma']$ for some $k \in \mathbb{N}$. We assume $\tilde{w}(Z, X) = Z \cdot \tilde{eq}(X, \alpha)$ for some $\alpha \in \mathbb{F}^m$, which is the case of interest to us. Let ρ to denote the rate of the Reed-Solomon code, i.e., $\rho = 2^m / |L|$. Let $f : L \rightarrow \mathbb{F}$ be a function and let $\delta \in (0, 1 - \sqrt{\rho})$ be a proximity parameter. The claim $f \in \text{CRS}[\mathbb{F}, L, m, \tilde{w}, \sigma]$ is essentially a sum-check instance $\sum_{\mathbf{b} \in \{0,1\}^m} \tilde{w}(\tilde{f}(\mathbf{b}), \mathbf{b}) = \sigma$. Therefore, initially $\mathcal{P}$ and $\mathcal{V}$ run k rounds of sum-check protocol as below:

SumCheck Phase: The prover $\mathcal{P}$ and verifier $\mathcal{V}$ interact as follows:

- In round i for $i \in [k]$.
 - $\mathcal{P}$ computes:

$$h_i(X) = \sum_{\mathbf{b} \in \{0,1\}^{m-i}} \tilde{w}(f(\alpha_i, X, \mathbf{b}), \alpha_i, X, \mathbf{b})$$
 where $\alpha_i = (\alpha_1, \dots, \alpha_{i-1})$ denotes the vector of challenges sent in the first $i-1$ rounds.
 - $\mathcal{P}$ sends $h_i(X)$.
 - $\mathcal{V}$ checks: If $i > 1$, $\mathcal{V}$ checks $h_i(0) + h_i(1) = h_{i-1}(\alpha_{i-1})$. For $i = 1$, it checks $h_1(0) + h_1(1) = \sigma$.
 - $\mathcal{V}$ sends $\alpha_i \leftarrow \mathbb{F}$.
- The prover sends the function $f' = \text{fold}(f, (\alpha_1, \dots, \alpha_k))^6$.
- The reduced claim is now $\sum_{\mathbf{b} \in \{0,1\}^{m-k}} \tilde{w}(f'(\mathbf{b}), \alpha, \mathbf{b}) = h_k(\alpha_k)$ where $\alpha = (\alpha_1, \dots, \alpha_k)$.

In our presentation we consider only one iteration and $\mathcal{P}$ sends the function f' in full. In [5] f' is again sent as an oracle for subsequent iterations.

Query Phase: After $\mathcal{P}$ sends f' in the final step, the verifier makes queries to check consistency of f' and f , and that f' satisfies the evaluation claim at the end of the sum-check phase.

- $\mathcal{V}$ samples t random points $z_1, \dots, z_t \leftarrow L^{(2^k)}$ and queries $y_i = \llbracket \text{fold}(f, \alpha) \rrbracket(z_i)$ for $i \in [t]$. Note that the verifier simulates queries to $\text{fold}(f, \alpha)$ using 2^k queries to $\llbracket f \rrbracket$.
- $\mathcal{V}$ checks $\tilde{f}'(z_i) = y_i$ for all $i \in [t]$, where $\mathbf{z}_i = (z_i, z_i^2, \dots, z_i^{2^{m-k-1}})$.
- $\mathcal{V}$ checks reduced claim: $\sum_{\mathbf{b} \in \{0,1\}^{m-k}} \tilde{w}(f'(\mathbf{b}), \alpha, \mathbf{b}) = h_k(\alpha_k)$.

By replacing evaluation claims on $\tilde{f}'$ with equivalent sum-check instances and batching the $t+1$ resulting sum-check instances using uniformly sampled $\gamma \leftarrow \mathbb{F}$, $\mathcal{V}$'s checks reduce to the following inner product:

$$\sum_{\mathbf{b} \in \{0,1\}^{m-k}} \tilde{f}'(\mathbf{b}) \cdot \left(\tilde{eq}(\alpha, \mathbf{b}, \mathbf{z}) + \sum_{i=1}^t \gamma^i \cdot \tilde{eq}(\mathbf{b}, \mathbf{z}_i) \right) = \sigma'$$

Thus, $\mathcal{V}$'s checks can be done in $O(t \cdot 2^{m-k})$ $\mathbb{F}$ operations. The soundness of the sum-check steps follows from the followig Lemma based on [5, Lemma 2].

LEMMA C.1. *Let $C := \text{CRS}[\mathbb{F}, L, m, \tilde{w}, \sigma]$ be a constrained Reed-Solomon code with rate $\rho = 2^m / |L|$. Let $f : L \rightarrow \mathbb{F}$ be a function such that $\Delta(f, C) > \delta$ for some $\delta \in (0, 1 - \sqrt{\rho})$. For $k < m$, let $h_1, \dots, h_k$ be $\mathcal{P}$'s polynomials in $\mathbb{F}^{<3}[X]$ and $\alpha_1, \dots, \alpha_k$ be $\mathcal{V}$'s challenges in an accepting transcript for the sum-check phase. Then $\Delta(\text{fold}(f, \alpha), C_k) > \delta$ with probability $\geq 1 - \text{poly}(2^m, 1/\rho)/|\mathbb{F}|$.*

⁶The prover only needs to send the message positions of the codeword, as they uniquely specify the function

where $C_k := \text{CRS}[\mathbb{F}, L^{(2^k)}, m - k, \tilde{w}_k, \sigma_k]$ with $\tilde{w}_k(Z, \mathbf{X}) = Z \cdot \tilde{w}(\alpha_1, \dots, \alpha_k, \mathbf{X})$ and $\sigma_k = h_k(\alpha_k)$.

Intuitively, Lemma C.1 implies that if prover's initial function is δ -far from C , its folding remains δ -far from the constrained code determined by sum-check claim in later rounds. After k rounds, since the prover is forced to send a codeword in C_k , the consistency checks succeed with probability at most $(1 - \delta)^t$. Thus we have:

LEMMA C.2. *For all $\delta \in (0, 1 - \sqrt{\rho})$, if $\Delta(f, \text{CRS}[\mathbb{F}, L, m, \tilde{w}, \sigma]) > \delta$, then the verifier accepts in the above protocol with probability at most $(1 - \delta)^t + \frac{\text{poly}(\rho^{-1}, m)}{|\mathbb{F}|}$.*

In [5], the above result was proved under the assumption that Reed-Solomon codes satisfy *mutual correlated agreement* up to $\delta < 1 - \sqrt{\rho}$. In [5], the authors only proved the assumption for $\delta < (1 - \rho)/2$; it has subsequently been proved for $\delta < 1 - \sqrt{\rho}$ in [34]. For honestly encoded oracles, the soundness of WHIR iteration follows from much simpler arguments.

Evaluation Claims on Honest Oracles: We can strengthen the soundness bound in Lemma C.2 when the oracle $\llbracket f \rrbracket$ is *known* to encode a valid codeword (e.g. it is output as part of one time pre-processing). In this case, the soundness of consistency check between f' and $\text{fold}(f, \alpha)$ improves to ρ^t , where t is the number of queries. We state this below:

LEMMA C.3. *For $f \in \text{RS}[\mathbb{F}, L, m]$ and $f \notin \text{CRS}[\mathbb{F}, L, m, \tilde{w}, \sigma]$, the verifier in WHIR iteration accepts with probability at most $\rho^t + O(k/|\mathbb{F}|)$.*

PROOF. We observe that if $f \in \text{RS}[\mathbb{F}, L, m]$, we have $\text{fold}(f, \alpha) \in \text{RS}[\mathbb{F}, L', m - k]$. Therefore, if $f' \neq \text{fold}(f, \alpha)$, we have $\Delta(\text{fold}(f, \alpha), f') \geq 1 - \rho$. Thus if $f' \neq \text{fold}(f, \alpha)$, the consistency checks succeed with probability at most ρ^t . If f' is honest, then the soundness follows from the soundness of the sum-check protocol which in this case is $O(k/|\mathbb{F}|)$. $\square$

The above result allows us to optimize proof sizes when querying trusted oracles.

C.2 Sum-Check over Group Polynomials

In this work, we consider polynomials over a group $\mathbb{G}$ of prime order p written additively. In [18], Bootle et al. considered generalization of sum-check protocol for polynomials over modules. We consider the special case when the module is a group $\mathbb{G}$ (viewed as a $\mathbb{Z}_p$ -module, in fact a $\mathbb{Z}_p$ -vector space). A univariate polynomial $\tilde{g}(X) \in \mathbb{G}[X]$ is given by $\tilde{g}(X) := \sum_{i=1}^d g_i X^{i-1}$ for some $g_1, \dots, g_d \in \mathbb{G}$. A multilinear polynomial $\tilde{g}(X_1, \dots, X_m) \in \mathbb{G}[X_1, \dots, X_m]$ is given by $\tilde{g}(X_1, \dots, X_m) = \sum_{\mathbf{b} \in \{0,1\}^m} g_{\mathbf{b}} \cdot \prod_{j=1}^m X_j^{b_j}$ for some $g_{\mathbf{b}} \in \mathbb{G}$. Note that the evaluations $\tilde{g}(\mathbf{a}) \in \mathbb{G}$ and $\tilde{g}(\mathbf{a}) \in \mathbb{G}$ for $\mathbf{a} \in \mathbb{F}^m$ are well-defined. We can also see multiplication of a field polynomial with a group polynomial extends naturally by just taking MSMs of the coefficients. By using the above discussion, we can define sum-check for group polynomials.

LEMMA C.4 ([18]). *Let $\mathbb{G}$ be a group of prime order p written additively. Let $\tilde{g}(X_1, \dots, X_m) \in \mathbb{G}[X_1, \dots, X_m]$ and $\tilde{f}(X_1, \dots, X_m) \in \mathbb{F}[X_1, \dots, X_m]$ be multilinear polynomials over the group $\mathbb{G}$ and field $\mathbb{F} = \mathbb{Z}_p$ respectively. Then, there exists an interactive protocol between*

a prover $\mathcal{P}$ and verifier $\mathcal{V}$ to verify the claim $\sum_{\mathbf{b} \in \{0,1\}^m} \tilde{g}(\mathbf{b}) \cdot \tilde{f}(\mathbf{b}) = \sigma$ for $\sigma \in \mathbb{G}$ where:

- $\mathcal{P}$ performs $O(2^m)$ group and field operations,
- $\mathcal{V}$ performs $O(m)$ group operations and field operations, in addition to one query each to polynomials $\tilde{g}$ and $\tilde{f}$.
- The protocol achieves perfect completeness and soundness error of $O(m/|\mathbb{F}|)$.

We note that the messages sent by the prover in the above protocol are group elements describing the univariate group polynomials.

D TitanSnark: Spartan with Titan

We introduce TitanSnark, based on MicroSpartan [51] PIOP (variant of the Spartan PIOP for R1CS from [45]), slightly redesigned for Titan PCS. In particular, we reduce size of auxiliary polynomial oracles at the cost of slightly increased sum-check communication. We provide detailed PIOP in Section E. Here, we give an overview of how Titan PCS is involved in the compilation of PIOP into a SNARK.

For $m, n \in \mathbb{N}$ and field $\mathbb{F}$, the relation $\mathcal{R1CS}_{m,n,\mathbb{F}}$ consists of tuples $(A, B, C; \mathbf{z})$ with $A, B, C \in \mathbb{F}^{m \times n}$ and $\mathbf{z} \in \mathbb{F}^n$ satisfying $A\mathbf{z} \circ B\mathbf{z} = C\mathbf{z}^T$. We consider A, B and C to be sparse matrices with $O(m + n)$ non-zero entries. Without loss of generality, we can assume $m = n$, which can be ensured by zero-padding vectors and matrices. We describe our variant of Spartan PIOP in Appendix E, which closely follows presentation in [45]. The PIOP in Appendix E requires $\mathcal{P}$ to provide following evaluations for $\log N$ -variate polynomials (where N is the total number of non-zero entries in A, B and C combined).

- Evaluations of index polynomials (trusted) $\tilde{row}, \tilde{col}, \tilde{val}_A, \tilde{val}_B$ and $\tilde{val}_C$ at $\mathbf{r} \in \mathbb{F}^{\log N}$.
- Evaluations of witness polynomial $\tilde{z}$ at $\mathbf{r}_y \in \mathbb{F}^{\log N}$.
- Evaluations of second round polynomials $\tilde{p}, \tilde{q}, \tilde{m}_r, \tilde{m}_c$ at $\mathbf{r} \in \mathbb{F}^{\log N}$.
- Evaluation of third round polynomial $\tilde{g}$ at $\mathbf{r}$.

In the above, polynomials $\tilde{z}, \tilde{m}_r$ and $\tilde{m}_c$ have “short” coefficients. We implement following standard optimisations to reduce proof size and verification complexity.

Oracle Aggregation: We aggregate four second round polynomials into a $(\log N + 2)$ -variate polynomial $\tilde{w}$. Then queries to constituent polynomials at $\mathbf{r}$ reduce to queries to $\tilde{w}$ at $(\mathbf{r}, 0, 0), \dots, (\mathbf{r}, 1, 1)$. We use the standard trick of reducing distinct queries to $\tilde{w}$ to a single query to $\tilde{w}$ at $\mathbf{r}' \in \mathbb{F}^{\log N + 2}$ via a degree-2 sumcheck [21]. The prover thus sends oracle $\llbracket \tilde{w} \rrbracket$ to the aggregate polynomial $\tilde{w}$.

Oracle Batching: In this case, we batch the claims $\tilde{f}_i(\mathbf{r}) = v_i, i \in [k]$ using a random linear combination to the claim $\sum_{i=1}^k \gamma^{i-1} \tilde{f}_i(\mathbf{r}) = \sum_{i=1}^k \gamma^{i-1} v_i$. We simulate queries to $\llbracket \sum_{i=1}^k \gamma^{i-1} \tilde{f}_i \rrbracket$ by querying each constituent oracle. Note that instead of sending partially evaluated polynomials $\tilde{a}_i(\mathbf{Y}) = \tilde{f}_i(\mathbf{r}_x, \mathbf{Y})$ for each $i \in [k]$, $\mathcal{P}$ can simply send the batched evaluation $\sum_{i=1}^k \gamma^{i-1} \tilde{a}_i(\mathbf{Y})$. This allows us to use just one instance of the CSP for evaluation of the partial polynomial. We use oracle batching for index polynomials $\tilde{row}, \tilde{col}, \tilde{val}_A, \tilde{val}_B$ and $\tilde{val}_C$. We also note that querying trusted polynomials incurs less queries, as soundness for each query is $\approx \rho$ (the rate of the

⁷It is customary to consider $\mathbf{z} = (1, \mathbf{x}, \mathbf{w})$ with $\mathbf{x}$ denoting public inputs. We ignore this decomposition for simplicity

RS encoding). We also batch oracles $\llbracket \tilde{z} \rrbracket$ and $\llbracket \tilde{q} \rrbracket$. Since these are queried on distinct points, we first invoke a degree-2 sumcheck to reduce the queries to the same point.

In Table 4, we evaluate the performance of TitanSnark along with Spartan_{DL} [45]. For Spartan_{DL}, benchmarks we use the available implementation at [40]. We use identical method of generating synthetic R1CS matrices, as in the reference implementation [40]. We note that the implementation [40] implements a zero-knowledge variant which incurs additional prover overheads. The implementations also use different curves (Ristretto vs Pasta), so the comparison (specifically of prover time) is not entirely apples-to-apples. However, for larger polynomial sizes, the asymptotic advantage of underlying PCS Titan ($\sqrt[3]{n}$ vs $\sqrt{n}$) in verification and proof size starts to manifest itself. For TitanSnark, the most expensive operation is committing to auxiliary polynomials with large coefficients, followed by the computing sumcheck messages and evaluation proofs. The proof sizes are relatively short in TitanSnark, as group oracles for index polynomials can be locally stored by the verifier (being $O(\sqrt{n})$ in size), and thus queries to them do not incur communication. The overhead of evaluations of index polynomials is also very low in TitanSnark, being $O(\sqrt{n})$, largely using MSMs.

Table 4: Performance comparison of Spartan variants for R1CS instances of size $N = 2^{20}, 2^{21}, 2^{22}$ and 2^{23} . * The implementation of Spartan_{DL} is zero-knowledge, while that of TitanSnark is not.

N	Scheme	Prover* (s)	Verifier (ms)	Proof Size (KB)
2^{20}	Spartan _{DL}	32.5	93	180
	TitanSnark	39	105	93
2^{21}	Spartan _{DL}	67	121	252
	TitanSnark	68	118	102
2^{22}	Spartan _{DL}	124	173	290
	TitanSnark	138	124	120
2^{23}	Spartan _{DL}	245	230	429
	TitanSnark	240	139	137

E TitanSnark: Details

Let A, B and C be $m \times n$ matrices over the field $\mathbb{F}$ defining the R1CS relation as in Section D. Formally, let n_A, n_B and n_C be number of non-zero entries in matrices A, B and C respectively. Let $N = n_A + n_B + n_C$ and let M be the matrix obtained by row-wise concatenation of A, B and C . Let $\{(\text{row}_i, \text{col}_i, \text{val}_i)\}_{i=1}^N$ be a canonical ordering of the non-zero tuples in the sparse representation of M , where we assume that first n_A tuples come from A , next n_B come from B and the remaining n_C come from C . Without loss of generality, we zero-pad matrices A, B and C to size $N \times N$. We define vectors $\text{row}, \text{col}, \text{val}_A, \text{val}_B, \text{val}_C \in \mathbb{F}^N$ as follows:

- For $i = 1, \dots, N$: $\text{row}[i] = \text{row}_i, \text{col}[i] = \text{col}_i$.
- For $i = 1, \dots, n_A$: $\text{val}_A[i] = \text{val}_i$.
- For $i = 1, \dots, n_B$: $\text{val}_B[i] = \text{val}_{i+n_A}$.
- For $i = 1, \dots, n_C$: $\text{val}_C[i] = \text{val}_{i+n_A+n_B}$.

For all the above vectors, the positions not explicitly specified are assumed to be 0. Let $\widetilde{\text{row}}, \widetilde{\text{col}}, \widetilde{\text{val}}_A, \widetilde{\text{val}}_B$ and $\widetilde{\text{val}}_C$ denote the MLEs

of above vectors. We note that for each $K \in \{A, B, C\}$, the MLE $\widetilde{K}$ of K is given by:

$$\begin{aligned} \widetilde{K}(\mathbf{x}, \mathbf{y}) &= \sum_{i=1}^N \sum_{j=1}^N K(i, j) \cdot \widetilde{eq}(\mathbf{x}, \langle i \rangle) \cdot \widetilde{eq}(\mathbf{y}, \langle j \rangle) \\ &= \sum_{i=1}^N \text{val}_K[i] \cdot \widetilde{eq}(\mathbf{x}, \text{row}[i]) \cdot \widetilde{eq}(\mathbf{y}, \text{col}[j]) \end{aligned} \quad (18)$$

In the above, we identify $x \in \mathbb{F}$ with $\langle x \rangle$ to simplify notation. Let $\mathbf{z} \in \mathbb{F}^n$ be R1CS witness satisfying $A\mathbf{z} \circ B\mathbf{z} = C\mathbf{z}$. As outlined in [45, Section 5], the check $A\mathbf{z} \circ B\mathbf{z} = C\mathbf{z}$ reduces via the application of sum-check protocol to the oracle queries $\widetilde{A}(\mathbf{r}_x, \mathbf{r}_y), \widetilde{B}(\mathbf{r}_x, \mathbf{r}_y), \widetilde{C}(\mathbf{r}_x, \mathbf{r}_y)$ and $\widetilde{z}(\mathbf{r}_y)$ for some $\mathbf{r}_x, \mathbf{r}_y \in \mathbb{F}^{\log N}$. Let $\mathbf{T}_x$ and $\mathbf{T}_y$ denote tables of size N given by $\mathbf{T}_x[i] = \widetilde{eq}(\mathbf{r}_x, \langle i \rangle)$ and $\mathbf{T}_y[i] = \widetilde{eq}(\mathbf{r}_y, \langle i \rangle)$ for $i \in [N]$. Using, Equation (18), we can write:

$$\widetilde{K}(\mathbf{r}_x, \mathbf{r}_y) = \sum_{\mathbf{x} \in B_{\log N}} \widetilde{\text{val}}_K(\mathbf{x}) \cdot \widetilde{p}(\mathbf{x}) \cdot \widetilde{q}(\mathbf{x}) \text{ for } K \in \{A, B, C\} \quad (19)$$

where $\widetilde{p}$ and $\widetilde{q}$ are multilinear polynomials defined by $\widetilde{p}(\mathbf{x}) = \mathbf{T}_x[\widetilde{\text{row}}(\mathbf{x})]$ and $\widetilde{q}(\langle i \rangle) = \mathbf{T}_y[\widetilde{\text{col}}[i]]$. The correctness of lookups to compute polynomials $\widetilde{p}$ and $\widetilde{q}$ can be reduced to a sum-check involving auxiliary polynomial oracles corresponding to multiplicities of elements of the tables in the target vector. Using the approach of [33], the prover sends polynomial oracles $\llbracket \widetilde{m}_r \rrbracket, \llbracket \widetilde{m}_c \rrbracket$ and shows the equality of rational functions:

$$\begin{aligned} \sum_{\mathbf{x} \in B_{\log N}} \frac{\widetilde{m}_r(\mathbf{x})}{X + \widetilde{id}(\mathbf{x}) \cdot Y + \widetilde{eq}(\mathbf{x}, \mathbf{r}_x)} &= \sum_{\mathbf{x} \in B_{\log N}} \frac{1}{X + \widetilde{\text{row}}(\mathbf{x}) \cdot Y + \widetilde{p}(\mathbf{x})} \\ \sum_{\mathbf{x} \in B_{\log N}} \frac{\widetilde{m}_c(\mathbf{x})}{X + \widetilde{id}(\mathbf{x}) \cdot Y + \widetilde{eq}(\mathbf{x}, \mathbf{r}_y)} &= \sum_{\mathbf{x} \in B_{\log N}} \frac{1}{X + \widetilde{\text{col}}(\mathbf{x}) \cdot Y + \widetilde{q}(\mathbf{x})} \end{aligned}$$

The verifier probabilistically checks the above at $X = \alpha, Y = \beta$ for $\alpha, \beta \leftarrow \mathbb{F}$. The verifier sends $\gamma \leftarrow \mathbb{F}$ to batch the above checks. The prover sends oracle $\widetilde{g}$ to the polynomial interpolating the rational function

$$\widetilde{g}(\mathbf{x}) = \widetilde{m}_r(\mathbf{x})/\widetilde{d}_1(\mathbf{x}) - 1/\widetilde{d}_2(\mathbf{x}) + \gamma \widetilde{m}_c(\mathbf{x})/\widetilde{d}_3(\mathbf{x}) - \gamma/\widetilde{d}_4(\mathbf{x})$$

where $\widetilde{d}_1(\mathbf{x}), \widetilde{d}_2(\mathbf{x}), \widetilde{d}_3(\mathbf{x})$ and $\widetilde{d}_4(\mathbf{x})$ denote the four denominators after setting $X = \alpha$ and $Y = \beta$, i.e.,

$$\begin{aligned} \widetilde{d}_1(\mathbf{x}) &= \alpha + \beta \cdot \widetilde{id}(\mathbf{x}) + \widetilde{eq}(\mathbf{x}, \mathbf{r}_x), & \widetilde{d}_2(\mathbf{x}) &= \alpha + \beta \cdot \widetilde{\text{row}}(\mathbf{x}) + \widetilde{p}(\mathbf{x}) \\ \widetilde{d}_3(\mathbf{x}) &= \alpha + \beta \cdot \widetilde{id}(\mathbf{x}) + \widetilde{eq}(\mathbf{x}, \mathbf{r}_y), & \widetilde{d}_4(\mathbf{x}) &= \alpha + \beta \cdot \widetilde{\text{col}}(\mathbf{x}) + \widetilde{q}(\mathbf{x}) \end{aligned}$$

To check correctness of $\widetilde{g}$, the verifier sends another challenge $\eta \leftarrow \mathbb{F}^{\log N}$. The checks reduce to the following sum-check instances:

$$\begin{aligned} \sum_{\mathbf{x} \in B_{\log N}} \widetilde{eq}(\mathbf{x}, \eta) \cdot \left(\widetilde{t}(\mathbf{x}) - \widetilde{g}(\mathbf{x}) \widetilde{d}_1(\mathbf{x}) \widetilde{d}_2(\mathbf{x}) \widetilde{d}_3(\mathbf{x}) \widetilde{d}_4(\mathbf{x}) \right) &= 0 \\ \sum_{\mathbf{x} \in B_{\log N}} \widetilde{g}(\mathbf{x}) &= 0 \end{aligned} \quad (20)$$

where we define $\widetilde{t}(\mathbf{x}) = \widetilde{m}_r(\mathbf{x})\widetilde{d}_1(\mathbf{x}) - \widetilde{d}_2(\mathbf{x}) + \gamma \widetilde{m}_c(\mathbf{x})\widetilde{d}_3(\mathbf{x}) - \gamma \widetilde{d}_4(\mathbf{x})$ and $\widetilde{D}_i(\mathbf{x}) = \prod_{j \neq i} \widetilde{d}_j(\mathbf{x})$. Using a random challenge, all the sum-check instances in Equations (19), and (20) can be aggregated into a single sum-check instance of the form

$$\widetilde{G}(\widetilde{\text{row}}, \widetilde{\text{col}}, \widetilde{\text{val}}_A, \widetilde{\text{val}}_B, \widetilde{\text{val}}_C, \widetilde{z}, \widetilde{p}, \widetilde{q}, \widetilde{m}_r, \widetilde{m}_c, \widetilde{g}, \widetilde{id}, \widetilde{eq}(\cdot, \mathbf{r}_x), \widetilde{eq}(\cdot, \mathbf{r}_y)) = \sigma \quad (21)$$

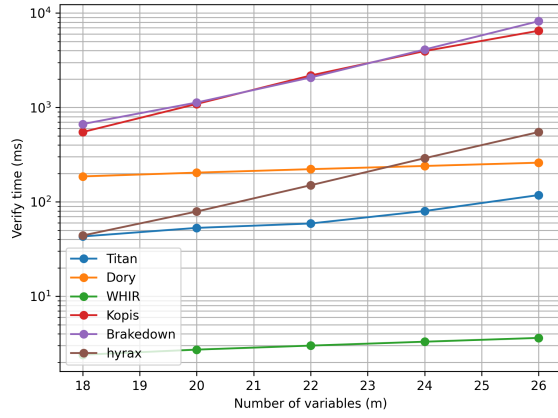


Figure 8: Verification time

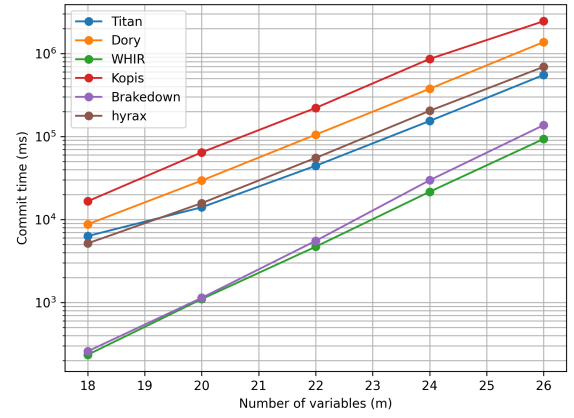


Figure 6: Commitment time

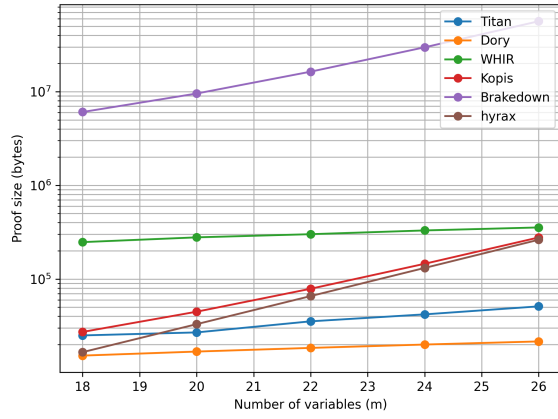


Figure 9: Proof size

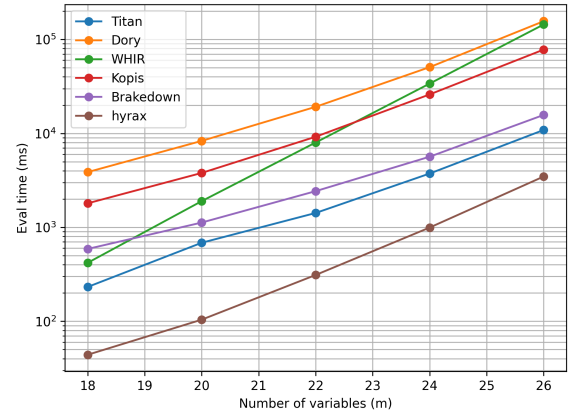


Figure 7: Evaluation time

where $\bar{G}$ is a multivariate polynomial of total degree ≤ 6 .

F Visual Comparison of Commitment Schemes

Figures 6, 7, 8, and 9 compare the commitment time, evaluation time, verification time, and proof size, respectively across all schemes as the number of variables m increases from 18 to 26.